Memoria técnica — Práctica 1: Ethereum y Solidity

Autoría: Alicia Jiajun Lorenzo Lourido

Fecha: 27/10/2026

Repositorio local: /mnt/c/Users/ajlor/Desktop/git/Ciberseguridad_Munics_clean/blockchain/TRD_Blockchain/Practica_1

Documento generado a partir de archivos presentes. No ejecuta contratos ni pruebas.

1. Introducción y organización

La práctica aborda Solidity y Ethereum mediante ejercicios progresivos: CryptoZombies y E1, tokens E2 y E4, documentación E3 y el proyecto PharmaChain Ethereum (E5). La memoria conserva trazabilidad hacia archivos reales.

Bloque	Contenido
E1 y CryptoZombies	Contratos, herencia, mappings, eventos y NFT
E2	TokenContract.sol
E3	README e imágenes
E4	Token OpenZeppelin
E5	Cadena de custodia, roles, tests, GasLab y benchmarks

Documentación: README

Práctica 1 — Ethereum y contratos inteligentes con Solidity

Asignatura: Tecnologías de Registro Distribuido y Blockchain\

Máster: Ciberseguridad\

Universidad: Universidade de Vigo\

Fecha de entrega: 27 de octubre de 2026

1. Introducción

Esta práctica tiene como objetivo comprender el funcionamiento de Ethereum y desarrollar contratos inteligentes utilizando Solidity.

Documentación: Practica_1/INVENTARIO_CODIGO.md

Inventario automático del código

Documento generado a partir de los archivos existentes.

`CryptoZombies/Lesson_1/zombiefactory.sol`

Contratos: `ZombieFactory`

Funciones: `_createZombie`, `_generateRandomDna`, `createRandomZombie`

Eventos: `NewZombie`

Estructuras: `Zombie`

2. E1 — Solidity y CryptoZombies

Documentación: E1/Readme.md

Práctica 1 · Ejercicio 1 — CryptoZombies y Solidity

> **Asignatura:** Blockchain · **Entorno:** Ethereum local con Hardhat · **Actividad:** _Solidity: Beginner to Intermediate Smart Contracts_

1. Objetivo de la práctica

Ethereum ofrece distintas herramientas para aprender, programar y experimentar con contratos inteligentes. En este ejercicio se utiliza **CryptoZombies**, una plataforma educativa interactiva creada por el equipo de **Loom Network** que introduce Solidity mediante la construcción progresiva de un juego de zombis cripto-coleccionables.

La actividad consiste en completar el itinerario **Solidity: Beginner to Intermediate Smart Contracts**, conservar el código resultante de las lecciones y publicarlo en el repositorio de GitHub/GitLab. Como complemento, se incorpora una **demostración local visual** para interactuar con los contratos desde un navegador y observar su comportamiento en una blockchain de pruebas.

Objetivos de aprendizaje

- Familiarizarse con la sintaxis y los tipos de datos de Solidity.

- Comprender estructuras, funciones, modificadores, eventos, herencia y control de acceso.

- Estudiar la creación y gestión de zombis como activos digitales y la relación entre cuentas y propietarios.

- Desplegar contratos en una red Ethereum local y ejecutar transacciones.

- Diferenciar operaciones de lectura (sin transacción) y escritura (con consumo de gas de prueba).

- Observar restricciones del contrato, como permisos, niveles y tiempos de espera.

erc721.sol

Extracto del archivo E1/erc721.sol (11 líneas en total).

```
 2
 3 contract ERC721 {
 4     event Transfer(address indexed _from, address indexed _to, uint256 indexed _tokenId);
 5     event Approval(address indexed _owner, address indexed _approved, uint256 indexed _tokenId
 6
 7     function balanceOf(address _owner) external view returns (uint256);
 8     function ownerOf(uint256 _tokenId) external view returns (address);
 9     function transferFrom(address _from, address _to, uint256 _tokenId) external payable;
10     function approve(address _approved, uint256 _tokenId) external payable;
11 }
```

ownable.sol

Extracto del archivo E1/ownable.sol (72 líneas en total).

```
 4  * @title Ownable
 5  * @dev The Ownable contract has an owner address, and provides basic authorization control
 6  * functions, this simplifies the implementation of "user permissions".
 7  */
 8 contract Ownable {
 9     address private _owner;
10
11     event OwnershipTransferred(address indexed previousOwner, address indexed newOwner);
12
13     /**
14      * @dev The Ownable constructor sets the original `owner` of the contract to the sender
15      * account.
16      */
17     constructor() internal {
18         _owner = msg.sender;
19         emit OwnershipTransferred(address(0), _owner);
20     }
21
24      */
25     function owner() public view returns (address) {
26         return _owner;
27     }
28
29     /**

31      */
32     modifier onlyOwner() {
33         require(isOwner());
34         _;
35     }
36
37     /**
38      * @return true if `msg.sender` is the owner of the contract.
39      */
40     function isOwner() public view returns (bool) {
41         return msg.sender == _owner;
42     }
43
44     /**
45      * @dev Allows the current owner to relinquish control of the contract.
```

```
46      * @notice Renouncing to ownership will leave the contract without an owner.
47      * It will not be possible to call the functions with the `onlyOwner`
48      * modifier anymore.
49      */
50     function renounceOwnership() public onlyOwner {
51         emit OwnershipTransferred(_owner, address(0));
52         _owner = address(0);
```

safemath.sol

Extracto del archivo E1/safemath.sol (112 líneas en total).

```
 4  * @title SafeMath
 5  * @dev Math operations with safety checks that throw on error
 6  */
 7 library SafeMath {
 8     /**
 9      * @dev Multiplies two numbers, throws on overflow.
10      */
11     function mul(uint256 a, uint256 b) internal pure returns (uint256) {
12         if (a == 0) {
13             return 0;
14         }
15         uint256 c = a * b;
22      */
23     function div(uint256 a, uint256 b) internal pure returns (uint256) {
24         // assert(b > 0); // Solidity automatically throws when dividing by 0
25         uint256 c = a / b;
26         // assert(a == b * c + a % b); // There is no case in which this doesn't hold
27         return c;
32      */
33     function sub(uint256 a, uint256 b) internal pure returns (uint256) {
34         assert(b <= a);
35         return a - b;
36     }
37

40      */
41     function add(uint256 a, uint256 b) internal pure returns (uint256) {
42         uint256 c = a + b;
43         assert(c >= a);
44         return c;
45     }
49  * @title SafeMath32
50  * @dev SafeMath library implemented for uint32
51  */
52 library SafeMath32 {
53     function mul(uint32 a, uint32 b) internal pure returns (uint32) {
54         if (a == 0) {
55             return 0;
56         }
57         uint32 c = a * b;
61
62     function div(uint32 a, uint32 b) internal pure returns (uint32) {
63         // assert(b > 0); // Solidity automatically throws when dividing by 0
64         uint32 c = a / b;
65         // assert(a == b * c + a % b); // There is no case in which this doesn't hold
66         return c;
68
69     function sub(uint32 a, uint32 b) internal pure returns (uint32) {
70         assert(b <= a);

71         return a - b;
72     }
73
74     function add(uint32 a, uint32 b) internal pure returns (uint32) {
75         uint32 c = a + b;
76         assert(c >= a);
77         return c;
78     }
```

zombieattack.sol

Extracto del archivo E1/zombieattack.sol (29 líneas en total).

```
 4
 5 contract ZombieAttack is ZombieHelper {
 6     uint randNonce = 0;
 7     uint attackVictoryProbability = 70;
 8
 9     function randMod(uint _modulus) internal returns (uint) {
```

```
10          randNonce = randNonce.add(1);
11          return uint(keccak256(abi.encodePacked(now, msg.sender, randNonce))) % _modulus;
12      }
13
14      function attack(uint _zombieId, uint _targetId) external onlyOwnerOf(_zombieId) {
15          Zombie storage myZombie = zombies[_zombieId];
16          Zombie storage enemyZombie = zombies[_targetId];
17          uint rand = randMod(100);
18          if (rand <= attackVictoryProbability) {
```

zombiefactory.sol

Extracto del archivo E1/zombiefactory.sol (49 líneas en total).

```
 5
 6 contract ZombieFactory is Ownable {
 7     using SafeMath for uint256;
 8     using SafeMath32 for uint32;
 9     using SafeMath16 for uint16;
10
11     event NewZombie(uint zombieId, string name, uint dna);
12
13     uint dnaDigits = 16;
14     uint dnaModulus = 10 ** dnaDigits;
15     uint cooldownTime = 1 days;
16
17     struct Zombie {
18         string name;
19         uint dna;
20         uint32 level;
21         uint32 readyTime;
30
31     function _createZombie(string _name, uint _dna) internal {
32         uint id = zombies.push(Zombie(_name, _dna, 1, uint32(now + cooldownTime), 0, 0)) - 1;
33         zombieToOwner[id] = msg.sender;
34         ownerZombieCount[msg.sender] = ownerZombieCount[msg.sender].add(1);
35         emit NewZombie(id, _name, _dna);
37

38     function _generateRandomDna(string _str) private view returns (uint) {
39         uint rand = uint(keccak256(abi.encodePacked(_str)));
40         return rand % dnaModulus;
41     }
42
43     function createRandomZombie(string _name) public {
44         require(ownerZombieCount[msg.sender] == 0);
45         uint randDna = _generateRandomDna(_name);
46         randDna = randDna - (randDna % 100);
47         _createZombie(_name, randDna);
```

zombiefeeding.sol

Extracto del archivo E1/zombiefeeding.sol (66 líneas en total).

```
 4
 5 contract KittyInterface {
 6     function getKitty(
 7         uint256 _id
 8     )
 9         external
10         view
24
25 contract ZombieFeeding is ZombieFactory {
26     KittyInterface kittyContract;
27
28     modifier onlyOwnerOf(uint _zombieId) {
29         require(msg.sender == zombieToOwner[_zombieId]);
30         _;
31     }
32
33     function setKittyContractAddress(address _address) external onlyOwner {
34         kittyContract = KittyInterface(_address);
35     }
36
37     function _triggerCooldown(Zombie storage _zombie) internal {
38         _zombie.readyTime = uint32(now + cooldownTime);
39     }
40

41     function _isReady(Zombie storage _zombie) internal view returns (bool) {
42         return (_zombie.readyTime <= now);
```

```
43      }
44
45      function feedAndMultiply(
46          uint _zombieId,
47          uint _targetDna,
48          string _species
49      ) internal onlyOwnerOf(_zombieId) {
60
61      function feedOnKitty(uint _zombieId, uint _kittyId) public {
62          uint kittyDna;
63          (, , , , , , , , , kittyDna) = kittyContract.getKitty(_kittyId);
64          feedAndMultiply(_zombieId, kittyDna, "kitty");
65      }
```

zombiehelper.sol

Extracto del archivo E1/zombiehelper.sol (52 líneas en total).

```
 4
 5 contract ZombieHelper is ZombieFeeding {
 6     uint levelUpFee = 0.001 ether;
 7
 8     modifier aboveLevel(uint _level, uint _zombieId) {
 9         require(zombies[_zombieId].level >= _level);
10         _;
11     }
12
13     function withdraw() external onlyOwner {
14         address _owner = owner();
15         _owner.transfer(address(this).balance);
16     }
17
18     function setLevelUpFee(uint _fee) external onlyOwner {
19         levelUpFee = _fee;
20     }
21
22     function levelUp(uint _zombieId) external payable {
23         require(msg.value == levelUpFee);
24         zombies[_zombieId].level = zombies[_zombieId].level.add(1);
25     }
26
27     function changeName(

28         uint _zombieId,
29         string _newName
30     ) external aboveLevel(2, _zombieId) onlyOwnerOf(_zombieId) {
31         zombies[_zombieId].name = _newName;
33
34     function changeDna(
35         uint _zombieId,
36         uint _newDna
37     ) external aboveLevel(20, _zombieId) onlyOwnerOf(_zombieId) {
38         zombies[_zombieId].dna = _newDna;
40
41     function getZombiesByOwner(address _owner) external view returns (uint[]) {
42         uint[] memory result = new uint[](ownerZombieCount[_owner]);
43         uint counter = 0;
44         for (uint i = 0; i < zombies.length; i++) {
45             if (zombieToOwner[i] == _owner) {
```

zombieownership.sol

Extracto del archivo E1/zombieownership.sol (36 líneas en total).

```
 6
 7 contract ZombieOwnership is ZombieAttack, ERC721 {
 8     using SafeMath for uint256;
 9
10     mapping(uint => address) zombieApprovals;
11
12     function balanceOf(address _owner) external view returns (uint256) {
13         return ownerZombieCount[_owner];
14     }
15
16     function ownerOf(uint256 _tokenId) external view returns (address) {
17         return zombieToOwner[_tokenId];
18     }
19
20     function _transfer(address _from, address _to, uint256 _tokenId) private {
21         ownerZombieCount[_to] = ownerZombieCount[_to].add(1);
22         ownerZombieCount[msg.sender] = ownerZombieCount[msg.sender].sub(1);
```

```
23         zombieToOwner[_tokenId] = _to;
24         emit Transfer(_from, _to, _tokenId);
26
27     function transferFrom(address _from, address _to, uint256 _tokenId) external payable {
28         require(zombieToOwner[_tokenId] == msg.sender || zombieApprovals[_tokenId] == msg.send
29         _transfer(_from, _to, _tokenId);
30     }

31
32     function approve(address _approved, uint256 _tokenId) external payable onlyOwnerOf(_tokenI
33         zombieApprovals[_tokenId] = _approved;
34         emit Approval(msg.sender, _approved, _tokenId);
35     }
36 }
```

2.1. CryptoZombies integrado en E1

Las lecciones de CryptoZombies se presentan como parte del ejercicio E1 y se relacionan con los contratos Solidity desarrollados.

CryptoZombies — Lesson_1

Archivos de código y documentación encontrados: 2.

Documentación: CryptoZombies/Lesson_1/Lesson1.md

CryptoZombies — Solidity: Beginner to Intermediate Smart Contracts

Ejercicio 1 — Prácticas de Blockchain

Curso: Máster Universitario en Ciberseguridad (MUNICS)

Asignatura: Blockchain

Actividad: Ejercicio 1 — CryptoZombies

Extracto del archivo CryptoZombies/Lesson_1/zombiefactory.sol (167 líneas en total).

```
 8
 9 contract ZombieFactory {
10     // ========================================================
11     // CAPÍTULO 13: EVENTOS
12     // ========================================================
13     // Se incorpora un evento para informar de la creación
22
23     event NewZombie(uint zombieId, string name, uint dna);
24
25     // ========================================================
26     // CAPÍTULO 3: VARIABLES DE ESTADO Y ENTEROS
27     // ========================================================
54
55     struct Zombie {
56         string name;
57         uint dna;
58     }
59
91
92     function _createZombie(string _name, uint _dna) private {
93         // Crea una estructura Zombie con nombre y ADN.
94         // push() la introduce al final del array.
95         //
96         // En Solidity 0.4.25, push() devuelve la nueva

124
```

CryptoZombies — Lesson_2

Archivos de código y documentación encontrados: 3.

Documentación: CryptoZombies/Lesson_2/Lesson2.md

Lección 2 — CryptoZombies: Zombies Attack Their Victims

Descripción

En esta segunda lección se amplía el contrato inteligente `ZombieFactory` para introducir propietarios, restricciones de acceso, herencia e interacción con contratos externos.

El objetivo principal consiste en permitir que un zombi se alimente de otra criatura para generar un nuevo zombi cuyo ADN sea una combinación de ambos.

Desarrollo por capítulos

Extracto del archivo CryptoZombies/Lesson_2/zombiefactory.sol (52 líneas en total).

```
 2
 3 contract ZombieFactory {
 4     // LECCION 1 - CAPITULO 13: Eventos
 5     event NewZombie(uint zombieId, string name, uint dna);
 6
 7     // LECCION 1: ADN y estructuras
 8     uint dnaDigits = 16;
 9     uint dnaModulus = 10 ** dnaDigits;
10
11     struct Zombie {
12         string name;
13         uint dna;
14     }
15
28     // ZombieFeeding, que heredara de ZombieFactory.
29     function _createZombie(string _name, uint _dna) internal {
30         uint id = zombies.push(Zombie(_name, _dna)) - 1;
31
32         // Asigna el zombi a quien ejecuta la transaccion.
33         zombieToOwner[id] = msg.sender;
38
39     function _generateRandomDna(string _str) private view returns (uint) {
40         uint rand = uint(keccak256(abi.encodePacked(_str)));
41         return rand % dnaModulus;

42     }
```

Extracto del archivo CryptoZombies/Lesson_2/zombiefeeding.sol (80 líneas en total).

```
 7 // Describe la funcion getKitty de CryptoKitties.
 8 contract KittyInterface {
 9     function getKitty(
10         uint256 _id
11     )
12         external
13         view
30 // accesibles de ZombieFactory.
31 contract ZombieFeeding is ZombieFactory {
32     // CAPITULO 11: Referencia a otro contrato.
33     KittyInterface kittyContract;
34
35     // CAPITULO 11: Configurar la direccion externa.
37     // el ejercicio original: no es segura en produccion.
38     function setKittyContractAddress(address _address) public {
39         kittyContract = KittyInterface(_address);
40     }
41
42     // CAPITULOS 7-9: Storage, ADN y funciones.
43     // CAPITULO 13: Condicionales.
44     function feedAndMultiply(uint _zombieId, uint _targetDna, string _species) public {
45         // CAPITULO 4: Comprobar el propietario.
46         require(msg.sender == zombieToOwner[_zombieId]);
47

48         // CAPITULO 7: storage apunta al zombi
```

CryptoZombies — Lesson_3

Archivos de código y documentación encontrados: 5.

Extracto del archivo CryptoZombies/Lesson_3/L3ownable.sol (82 líneas en total).

```
 4 * @title Ownable
 5 * @dev The Ownable contract has an owner address, and provides basic authorization control
 6 * functions, this simplifies the implementation of "user permissions".
 7
 8 * CryptoZombies - Leccion 3
 9 *
17
18 contract Ownable {
```

```
19      address private _owner;
20
21      event OwnershipTransferred(address indexed previousOwner, address indexed newOwner);
22
23      /**
24       * @dev The Ownable constructor sets the original `owner` of the contract to the sender
25       * account.
26       */
27      constructor() internal {
28          _owner = msg.sender;
29          emit OwnershipTransferred(address(0), _owner);
30      }
31
34       */
35      function owner() public view returns (address) {
36          return _owner;

37      }
```

Extracto del archivo CryptoZombies/Lesson_3/L3zombiefactory.sol (52 líneas en total).

```
14
15 contract ZombieFactory is Ownable {
16      event NewZombie(uint zombieId, string name, uint dna);
17
18      uint dnaDigits = 16;
19      uint dnaModulus = 10 ** dnaDigits;
20      uint cooldownTime = 1 days;
21
22      struct Zombie {
23          string name;
24          uint dna;
25          uint32 level;
26          uint32 readyTime;
33
34      function _createZombie(string _name, uint _dna) internal {
35          uint id = zombies.push(Zombie(_name, _dna, 1, uint32(now + cooldownTime))) - 1;
36          zombieToOwner[id] = msg.sender;
37          ownerZombieCount[msg.sender]++;
38          emit NewZombie(id, _name, _dna);
40
41      function _generateRandomDna(string _str) private view returns (uint) {
42          uint rand = uint(keccak256(abi.encodePacked(_str)));
43          return rand % dnaModulus;
44      }

45
```

Extracto del archivo CryptoZombies/Lesson_3/L3zombiefeeding.sol (64 líneas en total).

```
11
12 contract KittyInterface {
13      function getKitty(
14          uint256 _id
15      )
16          external
17          view
40
41 contract ZombieFeeding is ZombieFactory {
42      KittyInterface kittyContract;
43
44      function setKittyContractAddress(address _address) external onlyOwner {
45          kittyContract = KittyInterface(_address);
46      }
47
48      function feedAndMultiply(uint _zombieId, uint _targetDna, string _species) public {
49          require(msg.sender == zombieToOwner[_zombieId]);
50          Zombie storage myZombie = zombies[_zombieId];
51          _targetDna = _targetDna % dnaModulus;
52          uint newDna = (myZombie.dna + _targetDna) / 2;
58
59      function feedOnKitty(uint _zombieId, uint _kittyId) public {
60          uint kittyDna;
61          (, , , , , , , , , kittyDna) = kittyContract.getKitty(_kittyId);

62          feedAndMultiply(_zombieId, kittyDna, "kitty");
```

Extracto del archivo CryptoZombies/Lesson_3/L3zombiehelper.sol (41 líneas en total).

```
13
14 contract ZombieHelper is ZombieFeeding {
15      modifier aboveLevel(uint _level, uint _zombieId) {
```

```
16          require(zombies[_zombieId].level >= _level);
17          _;
18      }
19
20      function changeName(uint _zombieId, string _newName) external aboveLevel(2, _zombieId) {
21          require(msg.sender == zombieToOwner[_zombieId]);
22          zombies[_zombieId].name = _newName;
23      }
24
25      function changeDna(uint _zombieId, uint _newDna) external aboveLevel(20, _zombieId) {
26          require(msg.sender == zombieToOwner[_zombieId]);
27          zombies[_zombieId].dna = _newDna;
28      }
29
30      function getZombiesByOwner(address _owner) external view returns (uint[]) {
31          uint[] memory result = new uint[](ownerZombieCount[_owner]);
32          uint counter = 0;
33          for (uint i = 0; i < zombies.length; i++) {
34              if (zombieToOwner[i] == _owner) {
```

Documentación: CryptoZombies/Lesson_3/Lesson3.md

CryptoZombies — Lección 3: Zombis que luchan

1. Objetivo

La Lección 3 amplía los contratos desarrollados en las dos primeras lecciones de CryptoZombies, introduciendo mecanismos de administración, control de acceso, niveles, tiempos de espera y consultas sobre los zombis de un usuario.

El objetivo es comprender cómo construir contratos inteligentes mediante herencia, modificadores y estructuras de almacenamiento.

Versión de Solidity: `^0.4.25`.

CryptoZombies — Lesson_4

Archivos de código y documentación encontrados: 6.

Extracto del archivo CryptoZombies/Lesson_4/L4ownable.sol (94 líneas en total).

```
10
11 contract Ownable {
12      // LECCION 3: Variables de tipo address
13      // Guarda la direccion del administrador.
14      // private impide acceder directamente desde
15      // otros contratos.
22      // propietario anterior y nuevo propietario.
23      event OwnershipTransferred(address indexed previousOwner, address indexed newOwner);
24
25      // LECCION 3: Constructores
26      // Se ejecuta durante el despliegue.
27      //
31      // internal: Ownable se utiliza mediante herencia.
32      constructor() internal {
33          _owner = msg.sender;
34
35          // Registra la asignacion inicial.
36          // address(0) indica que no habia propietario.
41      // view significa que no modifica el estado.
42      function owner() public view returns (address) {
43          return _owner;
44      }
45
46      // LECCION 3: Modificadores

53      // de la funcion que utiliza el modificador.
```

Extracto del archivo CryptoZombies/Lesson_4/L4zombieattack.sol (111 líneas en total).

```
 9 // Este es el contrato principal que se despliega.
10 contract ZombieAttack is ZombieHelper {
11      // LECCION 4 - CAPITULO 3: Numeros aleatorios
12      // Contador que cambia cada vez que se
13      // solicita un resultado pseudoaleatorio.
14      uint randNonce = 0;
26      // No utiliza view porque modifica randNonce.
27      function randMod(uint _modulus) internal returns (uint) {
```

```
28         // Incrementa el contador.
29         randNonce++;
30
31         // LECCION 1: keccak256 y abi.encodePacked
53     // propietario del atacante.
54     function attack(uint _zombieId, uint _targetId) external ownerOf(_zombieId) {
55         // LECCION 1: Structs
56         // storage permite modificar directamente
57         // los zombis almacenados en la blockchain.
58         Zombie storage myZombie = zombies[_zombieId];
```

Extracto del archivo CryptoZombies/Lesson_4/L4zombiefactory.sol (120 líneas en total).

```
 7
 8 contract ZombieFactory is Ownable {
 9     // LECCION 1 - CAPITULO 13: Eventos
10     // Notifica la creacion de un nuevo zombi.
11     event NewZombie(uint zombieId, string name, uint dna);
12
13     // LECCION 1: ADN y operaciones aritmeticas
14     // Cada zombi tiene un ADN de hasta 16 digitos.
15     uint dnaDigits = 16;
23     // Agrupamos los atributos de cada zombi.
24     struct Zombie {
25         // LECCION 1: Nombre y ADN.
26         string name;
27         uint dna;
28         // LECCION 3: Niveles y tiempo de espera.
59     // y los demas contratos derivados creen zombis.
60     function _createZombie(string _name, uint _dna) internal {
61         // LECCION 1: Arrays y estructuras
62         // push inserta un nuevo zombi.
63         //
64         // LECCION 3:
95     // No es una fuente de aleatoriedad segura.
96     function _generateRandomDna(string _str) private view returns (uint) {
97         uint rand = uint(keccak256(abi.encodePacked(_str)));

98         return rand % dnaModulus;
```

Extracto del archivo CryptoZombies/Lesson_4/L4zombiefeeding.sol (142 líneas en total).

```
 9 // de un CryptoKitty sin implementar su contrato.
10 contract KittyInterface {
11     // LECCION 2: Funciones externas
12     // external: se invoca desde otro contrato.
13     // view: consulta datos sin modificarlos.
14     //
16     // El ultimo contiene los genes del gato.
17     function getKitty(
18         uint256 _id
19     )
20         external
21         view
38 // variables de ZombieFactory y Ownable.
39 contract ZombieFeeding is ZombieFactory {
40     // LECCION 2: Referencias a contratos externos
41     // Almacena la interfaz del contrato CryptoKitties.
42     KittyInterface kittyContract;
43
49     // changeName, changeDna y attack.
50     modifier ownerOf(uint _zombieId) {
51         require(msg.sender == zombieToOwner[_zombieId]);
52         _;
53     }
54

57     // la direccion del contrato CryptoKitties.
```

Extracto del archivo CryptoZombies/Lesson_4/L4zombiehelper.sol (123 líneas en total).

```
 7
 8 contract ZombieHelper is ZombieFeeding {
 9     // LECCION 4 - CAPITULO 1: Funciones payable
10     // Precio inicial para aumentar un nivel.
11     //
12     // 0.001 ether equivale a 10^15 wei.
20     // Si no alcanza el nivel, la operacion revierte.
21     modifier aboveLevel(uint _level, uint _zombieId) {
22         require(zombies[_zombieId].level >= _level);
23         _;
24     }
```

```
25
30      // retire el dinero del contrato.
31      function withdraw() external onlyOwner {
32          // LECCION 3: Ownable
33          // Consulta quien administra el contrato.
34          address _owner = owner();
35
48      // Solo el administrador puede cambiarlo.
49      function setLevelUpFee(uint _fee) external onlyOwner {
50          levelUpFee = _fee;
51      }
52
53      // LECCION 4 - CAPITULO 1: Funciones payable

57      // junto con la transaccion.
```

Documentación: CryptoZombies/Lesson_4/lesson4.md

CryptoZombies — Lección 4: Batallas entre zombis

1. Introducción

La Lección 4 continúa el desarrollo del juego CryptoZombies utilizando Solidity `^0.4.25`.

Su objetivo es incorporar un sistema de combate entre zombis y mecanismos de pago mediante ETH, reutilizando las funcionalidades desarrolladas en las tres primeras lecciones.

El trabajo permite comprender cómo se gestionan pagos, permisos, estados persistentes y decisiones pseudoaleatorias en contratos inteligentes.

CryptoZombies — Lesson_5

Archivos de código y documentación encontrados: 9.

Extracto del archivo CryptoZombies/Lesson_5/L5erc721.sol (17 líneas en total).

```
 3 // LECCION 5 - CAPITULOS 1-2: interfaz de token no fungible; cada tokenId representa un zombi
 4 contract Lesson5ERC721 {
 5     // LECCION 5 - CAPITULO 2: evento que notifica cambios de propietario; indexed permite fil
 6     event Transfer(address indexed _from, address indexed _to, uint256 indexed _tokenId);
 7     // LECCION 5 - CAPITULO 2: evento que registra a quien se autoriza a transferir un token.
 8     event Approval(address indexed _owner, address indexed _approved, uint256 indexed _tokenId
 9     // LECCION 5 - CAPITULO 3: cantidad de zombis que posee una direccion.
10     function balanceOf(address _owner) external view returns (uint256);
11     // LECCION 5 - CAPITULO 3: obtiene el propietario de un tokenId.
12     function ownerOf(uint256 _tokenId) external view returns (address);
13     // LECCION 5 - CAPITULO 4: declara el mecanismo de transferencia; aqui no se implementa.
14     function transferFrom(address _from, address _to, uint256 _tokenId) external payable;
15     // LECCION 5 - CAPITULOS 6-7: declara la autorizacion de un tercero para transferir.
16     function approve(address _approved, uint256 _tokenId) external payable;
17 }
```

Extracto del archivo CryptoZombies/Lesson_5/L5ownable.sol (82 líneas en total).

```
 4  * @title Lesson5Ownable
 5  * @dev The Lesson5Ownable contract has an owner address, and provides basic authorization con
 6  * functions, this simplifies the implementation of "user permissions".
 7  */
 8 // LECCION 3 (REUTILIZADA EN LA 5): el propietario de este contrato es el administrador de la
 9 contract Lesson5Ownable {
10     // LECCION 3: direccion del administrador guardada de forma privada.
11     address private _owner;
12
13     // LECCION 3: registro auditable cuando cambia la administracion.
14     event OwnershipTransferred(address indexed previousOwner, address indexed newOwner);
15
16     /**
17      * @dev The Lesson5Ownable constructor sets the original `owner` of the contract to the se
18      * account.
19      */
20     // LECCION 3: el desplegador queda designado como administrador inicial.
21     constructor() internal {
22         _owner = msg.sender;
23         emit OwnershipTransferred(address(0), _owner);
24     }
25
29     // LECCION 3: consulta publica de la direccion administradora.
30     function owner() public view returns (address) {
```

```
31         return _owner;
```

Extracto del archivo CryptoZombies/Lesson_5/L5safemath.sol (104 líneas en total).

```
 4  * @title Lesson5SafeMath
 5  * @dev Math operations with safety checks that throw on error
 6  */
 7 // LECCION 5 - CAPITULOS 9-10: biblioteca para uint256; evita overflows y underflows en Solidi
 8 library Lesson5SafeMath {
 9     /** @dev Multiplies two numbers, throws on overflow. */
10     // LECCION 5 - CAPITULO 9: multiplica y verifica que no hubo overflow usando division inve
11     function mul(uint256 a, uint256 b) internal pure returns (uint256) {
12         if (a == 0) {
13             return 0;
14         }
15         uint256 c = a * b;
20     // LECCION 5 - CAPITULO 9: division entera; Solidity revierte si el divisor es cero.
21     function div(uint256 a, uint256 b) internal pure returns (uint256) {
22         // assert(b > 0); // Solidity automatically throws when dividing by 0
23         uint256 c = a / b;
24         // assert(a == b * c + a % b); // There is no case in which this doesn't hold
25         return c;
28     // LECCION 5 - CAPITULO 9: assert(b <= a) impide resultados negativos en enteros sin signo
29     function sub(uint256 a, uint256 b) internal pure returns (uint256) {
30         assert(b <= a);
31         return a - b;
32     }
33     /** @dev Adds two numbers, throws on overflow. */

34     // LECCION 5 - CAPITULO 9: assert(c >= a) detecta el desbordamiento de suma.
```

Extracto del archivo CryptoZombies/Lesson_5/L5zombieattack.sol (36 líneas en total).

```
 5 // LECCION 5 - CAPITULO 12: mantiene los combates y sustituye los incrementos por SafeMath.
 6 contract Lesson5ZombieAttack is Lesson5ZombieHelper {
 7     // LECCION 4: contador usado para variar el hash de resultados pseudoaleatorios.
 8     uint randNonce = 0;
 9     // LECCION 4: umbral de victoria; con <= 70 hay 71 resultados ganadores entre 0 y 99.
10     uint attackVictoryProbability = 70;
12     // LECCION 5 - CAPITULO 12: incrementa randNonce con SafeMath; la aleatoriedad sigue siend
13     function randMod(uint _modulus) internal returns (uint) {
14         randNonce = randNonce.add(1);
15         return uint(keccak256(abi.encodePacked(now, msg.sender, randNonce))) % _modulus;
16     }
17
18     // LECCION 5 - CAPITULO 12: protege las estadisticas y el nivel frente a desbordamientos.
19     function attack(uint _zombieId, uint _targetId) external onlyOwnerOf(_zombieId) {
20         Zombie storage myZombie = zombies[_zombieId];
21         Zombie storage enemyZombie = zombies[_targetId];
22         uint rand = randMod(100);
23         // LECCION 4: victoria: suma victoria/nivel, registra derrota rival y genera descenden
```

Extracto del archivo CryptoZombies/Lesson_5/L5zombiefactory.sol (65 líneas en total).

```
 6 // LECCIONES 1-5: fabrica base que conserva el estado de zombis y hereda el control administra
 7 contract Lesson5ZombieFactory is Lesson5Ownable {
 8     // LECCION 5 - CAPITULO 10: permite escribir numero.add(1) para uint256.
 9     using Lesson5SafeMath for uint256;
10     // LECCION 5 - CAPITULO 11: habilita operaciones seguras para uint32.
11     using Lesson5SafeMath32 for uint32;
15     // LECCION 1: evento emitido cada vez que se crea un zombi.
16     event NewZombie(uint zombieId, string name, uint dna);
17
18     // LECCION 1: longitud del ADN representado como numero.
19     uint dnaDigits = 16;
20     uint dnaModulus = 10 ** dnaDigits;
24     // LECCIONES 1-4: estructura con ADN, nivel, cooldown y estadisticas.
25     struct Zombie {
26         string name;
27         uint dna;
28         uint32 level;
29         uint32 readyTime;
42     // LECCION 5 - CAPITULO 11: alta del zombi y actualizacion segura del saldo del creador.
43     function _createZombie(string _name, uint _dna) internal {
44         uint id = zombies.push(Zombie(_name, _dna, 1, uint32(now + cooldownTime), 0, 0)) - 1;
45         zombieToOwner[id] = msg.sender;
46         // LECCION 5 - CAPITULO 11: SafeMath impide desbordamiento del contador de propiedad.
47         ownerZombieCount[msg.sender] = ownerZombieCount[msg.sender].add(1);

51     // LECCION 1: ADN derivado del nombre; no es aleatoriedad criptografica.
```

Extracto del archivo CryptoZombies/Lesson_5/L5zombiefeeding.sol (75 líneas en total).

```
// LECCION 2: interfaz para consultar ADN de un CryptoKitty externo.
contract Lesson5KittyInterface {
    function getKitty(
        uint256 _id
    )
        external
        view
// LECCIONES 2 Y 4: alimentacion y reproduccion con comprobacion de propiedad y cooldown.
contract Lesson5ZombieFeeding is Lesson5ZombieFactory {
    // LECCION 2: referencia a contrato externo de gatos.
    Lesson5KittyInterface kittyContract;

    // LECCION 4: comprueba propiedad del zombi; distinto de onlyOwner, que protege la adminis
    modifier onlyOwnerOf(uint _zombieId) {
        require(msg.sender == zombieToOwner[_zombieId]);
        _;
    }

    // LECCION 2: solo el administrador configura la direccion de CryptoKitties.
    function setKittyContractAddress(address _address) external onlyOwner {
        kittyContract = Lesson5KittyInterface(_address);
    }

    // LECCION 4: actualiza readyTime al momento actual mas un dia.

    function _triggerCooldown(Zombie storage _zombie) internal {
```

Extracto del archivo CryptoZombies/Lesson_5/L5zombiehelper.sol (62 líneas en total).

```
// LECCIONES 3-5: hereda utilidades de niveles, cambios de ADN/nombre y pagos.
contract Lesson5ZombieHelper is Lesson5ZombieFeeding {
    // LECCION 4: precio por subir de nivel, expresado en ether.
    uint levelUpFee = 0.001 ether;

    // LECCION 3: verifica el nivel minimo exigido.
    modifier aboveLevel(uint _level, uint _zombieId) {
        require(zombies[_zombieId].level >= _level);
        _;
    }

    // LECCION 4: solo el administrador retira el saldo del contrato.
    function withdraw() external onlyOwner {
        address _owner = owner();
        _owner.transfer(address(this).balance);
    }

    // LECCION 4: tarifa configurable por el administrador.
    function setLevelUpFee(uint _fee) external onlyOwner {
        levelUpFee = _fee;
    }

    // LECCION 5 - CAPITULO 12: subida de nivel con .add(1) de SafeMath32; la funcion original
    function levelUp(uint _zombieId) external payable {

        require(msg.value == levelUpFee);
```

Extracto del archivo CryptoZombies/Lesson_5/L5zombieownership.sol (51 líneas en total).

```
// LECCION 5 - CAPITULO 2: contrato final que combina juego y ERC721 mediante herencia multipl
contract Lesson5ZombieOwnership is Lesson5ZombieAttack, Lesson5ERC721 {
    // LECCION 5 - CAPITULO 10: operaciones seguras para balances ERC721.
    using Lesson5SafeMath for uint256;

    // LECCION 5 - CAPITULOS 6-7: autorizado por tokenId; el mapping no es publico.
    // LECCION 5 - CAPITULO 3: devuelve el saldo ERC721 del propietario.
    function balanceOf(address _owner) external view returns (uint256) {
        return ownerZombieCount[_owner];
    }

    // LECCION 5 - CAPITULO 3: resuelve el tokenId al propietario del zombi.
    function ownerOf(uint256 _tokenId) external view returns (address) {
        return zombieToOwner[_tokenId];
    }

    // LECCION 5 - CAPITULOS 4-5: cambia saldos, propietario y emite Transfer. ATENCION: el or
    function _transfer(address _from, address _to, uint256 _tokenId) private {
        // LECCION 5 - CAPITULO 10: incrementa con SafeMath el balance del receptor.
        ownerZombieCount[_to] = ownerZombieCount[_to].add(1);
        // LECCION 5 - CAPITULO 10: el original resta al emisor de la llamada; un tercero apro
        ownerZombieCount[msg.sender] = ownerZombieCount[msg.sender].sub(1);
    // LECCION 5 - CAPITULO 5: propietario o aprobado puede invocar; el original NO verifica q
    function transferFrom(address _from, address _to, uint256 _tokenId) external payable {
```

```
39         // LECCION 5 - CAPITULO 5: comprueba propietario o aprobado, pero no valida el paramet
```

CryptoZombies — Lesson_6

Archivos de código y documentación encontrados: 9.

Extracto del archivo CryptoZombies/Lesson_6/contracts/L6erc721.sol (17 líneas en total).

```
 3 // LECCION 5 - CAPITULOS 1-2: interfaz de token no fungible; cada tokenId representa un zombi
 4 contract Lesson6ERC721 {
 5     // LECCION 5 - CAPITULO 2: evento que notifica cambios de propietario; indexed permite fil
 6     event Transfer(address indexed _from, address indexed _to, uint256 indexed _tokenId);
 7     // LECCION 5 - CAPITULO 2: evento que registra a quien se autoriza a transferir un token.
 8     event Approval(address indexed _owner, address indexed _approved, uint256 indexed _tokenId
 9     // LECCION 5 - CAPITULO 3: cantidad de zombis que posee una direccion.
10     function balanceOf(address _owner) external view returns (uint256);
11     // LECCION 5 - CAPITULO 3: obtiene el propietario de un tokenId.
12     function ownerOf(uint256 _tokenId) external view returns (address);
13     // LECCION 5 - CAPITULO 4: declara el mecanismo de transferencia; aqui no se implementa.
14     function transferFrom(address _from, address _to, uint256 _tokenId) external payable;
15     // LECCION 5 - CAPITULOS 6-7: declara la autorizacion de un tercero para transferir.
16     function approve(address _approved, uint256 _tokenId) external payable;
17 }
```

Extracto del archivo CryptoZombies/Lesson_6/contracts/L6ownable.sol (82 líneas en total).

```
 4  * @title Lesson6Ownable
 5  * @dev The Lesson6Ownable contract has an owner address, and provides basic authorization con
 6  * functions, this simplifies the implementation of "user permissions".
 7  */
 8 // LECCION 3 (REUTILIZADA EN LA 5): el propietario de este contrato es el administrador de la
 9 contract Lesson6Ownable {
10     // LECCION 3: direccion del administrador guardada de forma privada.
11     address private _owner;
12
13     // LECCION 3: registro auditable cuando cambia la administracion.
14     event OwnershipTransferred(address indexed previousOwner, address indexed newOwner);
15
16     /**
17      * @dev The Lesson6Ownable constructor sets the original `owner` of the contract to the se
18      * account.
19      */
20     // LECCION 3: el desplegador queda designado como administrador inicial.
21     constructor() internal {
22         _owner = msg.sender;
23         emit OwnershipTransferred(address(0), _owner);
24     }
25
29     // LECCION 3: consulta publica de la direccion administradora.
30     function owner() public view returns (address) {

31         return _owner;
```

Extracto del archivo CryptoZombies/Lesson_6/contracts/L6safemath.sol (135 líneas en total).

```
 9  * @title Lesson6SafeMath
10  * @dev Math operations with safety checks that throw on error
11  */
12
13 // LECCION 6 - Arquitectura: componente desplegable o heredado por el contrato final.
14 library Lesson6SafeMath {
15     /**
16      * @dev Multiplies two numbers, throws on overflow.
17      */
18     // LECCION 6 - funcion heredada de lecciones anteriores; el frontend la invoca por ABI.
19     function mul(uint256 a, uint256 b) internal pure returns (uint256) {
20         if (a == 0) {
21             return 0;
22         }
23         uint256 c = a * b;
31     // LECCION 6 - funcion heredada de lecciones anteriores; el frontend la invoca por ABI.
32     function div(uint256 a, uint256 b) internal pure returns (uint256) {
33         // assert(b > 0); // Solidity automatically throws when dividing by 0
34         uint256 c = a / b;
35         // assert(a == b * c + a % b); // There is no case in which this doesn't hold
36         return c;
42     // LECCION 6 - funcion heredada de lecciones anteriores; el frontend la invoca por ABI.
43     function sub(uint256 a, uint256 b) internal pure returns (uint256) {
44         assert(b <= a);

45         return a - b;
```

Extracto del archivo CryptoZombies/Lesson_6/contracts/L6zombieattack.sol (39 líneas en total).

```
10 // LECCION 5 - CAPITULO 12: mantiene los combates y sustituye los incrementos por SafeMath.
11 contract Lesson6ZombieAttack is Lesson6ZombieHelper {
12     // LECCION 4: contador usado para variar el hash de resultados pseudoaleatorios.
13     uint randNonce = 0;
14     // LECCION 4: umbral de victoria; con <= 70 hay 71 resultados ganadores entre 0 y 99.
15     uint attackVictoryProbability = 70;
17     // LECCION 5 - CAPITULO 12: incrementa randNonce con SafeMath; la aleatoriedad sigue siend
18     function randMod(uint _modulus) internal returns (uint) {
19         randNonce = randNonce.add(1);
20         return uint(keccak256(abi.encodePacked(now, msg.sender, randNonce))) % _modulus;
21     }
22
23     // LECCION 6 - modifica estado: debe enviarse una transaccion.
24     function attack(uint _zombieId, uint _targetId) external onlyOwnerOf(_zombieId) {
25         Zombie storage myZombie = zombies[_zombieId];
26         Zombie storage enemyZombie = zombies[_targetId];
27         uint rand = randMod(100);
28         if (rand <= attackVictoryProbability) {
```

Extracto del archivo CryptoZombies/Lesson_6/contracts/L6zombiefactory.sol (65 líneas en total).

```
 6 // LECCIONES 1-5: fabrica base que conserva el estado de zombis y hereda el control administra
 7 contract Lesson6ZombieFactory is Lesson6Ownable {
 8     // LECCION 5 - CAPITULO 10: permite escribir numero.add(1) para uint256.
 9     using Lesson6SafeMath for uint256;
10     // LECCION 5 - CAPITULO 11: habilita operaciones seguras para uint32.
11     using Lesson6SafeMath32 for uint32;
15     // LECCION 1: evento emitido cada vez que se crea un zombi.
16     event NewZombie(uint zombieId, string name, uint dna);
17
18     // LECCION 1: longitud del ADN representado como numero.
19     uint dnaDigits = 16;
20     uint dnaModulus = 10 ** dnaDigits;
24     // LECCIONES 1-4: estructura con ADN, nivel, cooldown y estadisticas.
25     struct Zombie {
26         string name;
27         uint dna;
28         uint32 level;
29         uint32 readyTime;
42     // LECCION 5 - CAPITULO 11: alta del zombi y actualizacion segura del saldo del creador.
43     function _createZombie(string _name, uint _dna) internal {
44         uint id = zombies.push(Zombie(_name, _dna, 1, uint32(now + cooldownTime), 0, 0)) - 1;
45         zombieToOwner[id] = msg.sender;
46         // LECCION 5 - CAPITULO 11: SafeMath impide desbordamiento del contador de propiedad.
47         ownerZombieCount[msg.sender] = ownerZombieCount[msg.sender].add(1);

51     // LECCION 1: ADN derivado del nombre; no es aleatoriedad criptografica.
```

Extracto del archivo CryptoZombies/Lesson_6/contracts/L6zombiefeeding.sol (75 líneas en total).

```
 5 // LECCION 2: interfaz para consultar ADN de un CryptoKitty externo.
 6 contract Lesson6KittyInterface {
 7     function getKitty(
 8         uint256 _id
 9     )
10         external
11         view
26 // LECCIONES 2 Y 4: alimentacion y reproduccion con comprobacion de propiedad y cooldown.
27 contract Lesson6ZombieFeeding is Lesson6ZombieFactory {
28     // LECCION 2: referencia a contrato externo de gatos.
29     Lesson6KittyInterface kittyContract;
30
31     // LECCION 4: comprueba propiedad del zombi; distinto de onlyOwner, que protege la adminis
32     modifier onlyOwnerOf(uint _zombieId) {
33         require(msg.sender == zombieToOwner[_zombieId]);
34         _;
35     }
36
37     // LECCION 2: solo el administrador configura la direccion de CryptoKitties.
38     function setKittyContractAddress(address _address) external onlyOwner {
39         kittyContract = Lesson6KittyInterface(_address);
40     }
41
42     // LECCION 4: actualiza readyTime al momento actual mas un dia.

43     function _triggerCooldown(Zombie storage _zombie) internal {
```

Extracto del archivo CryptoZombies/Lesson_6/contracts/L6zombiehelper.sol (62 líneas en total).

```
 5 // LECCIONES 3-5: hereda utilidades de niveles, cambios de ADN/nombre y pagos.
```

```
contract Lesson6ZombieHelper is Lesson5ZombieFeeding {
    // LECCION 4: precio por subir de nivel, expresado en ether.
    uint levelUpFee = 0.001 ether;

    // LECCION 3: verifica el nivel minimo exigido.
    modifier aboveLevel(uint _level, uint _zombieId) {
        require(zombies[_zombieId].level >= _level);
        _;
    }

    // LECCION 4: solo el administrador retira el saldo del contrato.
    function withdraw() external onlyOwner {
        address _owner = owner();
        _owner.transfer(address(this).balance);
    }

    // LECCION 4: tarifa configurable por el administrador.
    function setLevelUpFee(uint _fee) external onlyOwner {
        levelUpFee = _fee;
    }

    // LECCION 5 - CAPITULO 12: subida de nivel con .add(1) de SafeMath32; la funcion original
    function levelUp(uint _zombieId) external payable {
        require(msg.value == levelUpFee);
```

Extracto del archivo CryptoZombies/Lesson_6/contracts/L6zombieownership.sol (48 líneas en total).

```
// LECCION 6 - Arquitectura: componente desplegable o heredado por el contrato final.
contract Lesson6ZombieOwnership is Lesson6ZombieAttack, Lesson6ERC721 {
    // LECCION 6 - SafeMath permite .add() y .sub() sin desbordamientos.
    using Lesson6SafeMath for uint256;

    mapping(uint => address) zombieApprovals;
    // LECCION 6 - consulta cuantos tokens posee una cuenta (call).
    function balanceOf(address _owner) external view returns (uint256) {
        return ownerZombieCount[_owner];
    }

    // LECCION 6 - consulta el propietario de un token (call).
    function ownerOf(uint256 _tokenId) external view returns (address) {
        return zombieToOwner[_tokenId];
    }

    // LECCION 6 - actualiza balances y emite Transfer; revisar comportamiento de msg.sender.
    function _transfer(address _from, address _to, uint256 _tokenId) private {
        ownerZombieCount[_to] = ownerZombieCount[_to].add(1);
        ownerZombieCount[msg.sender] = ownerZombieCount[msg.sender].sub(1);
        zombieToOwner[_tokenId] = _to;
        emit Transfer(_from, _to, _tokenId);
    // LECCION 6 - transfiere un token: cambia estado y emite Transfer.
    function transferFrom(address _from, address _to, uint256 _tokenId) external payable {
        require(zombieToOwner[_tokenId] == msg.sender || zombieApprovals[_tokenId] == msg.send
```

3. E2 — desarrollo y evidencias

Documentación: E2/Readme

TokenContract — Smart Contract en Solidity

1. Descripción

Implementación de un contrato inteligente en Solidity que permite gestionar y comprar tokens utilizando Ether (ETH).

La práctica se desarrolla en **Remix IDE**, utilizando Solidity `0.8.30` y una blockchain local simulada mediante Remix VM.

El contrato permite registrar usuarios, distribuir tokens y comprarlos a un precio fijo de **5 ETH por token**.

2. Objetivos

- Comprender las estructuras de datos `struct` y `mapping`.

- Gestionar permisos mediante modificadores (`onlyOwner`).

- Utilizar `msg.sender` para identificar usuarios.

- Implementar funciones `payable` para recibir Ether.

- Validar transacciones mediante `require`.

- Consultar el saldo del contrato inteligente.

TokenContract.sol

Extracto del archivo E2/TokenContract.sol (70 líneas en total).

```
 5
 6 contract TokenContract {
 7     address public owner;
 8     //precio fijo por token
 9     uint256 public constant TOKEN_PRICE = 5 ether;
10
11     // Eventos
12     event TokensPurchased(address indexed buyer, uint256 amount);
13
14     event TokensGifted(address indexed receiver, uint256 amount);
15
16     struct Receivers {
17         string name;
18         uint256 tokens;
19     }
20
22
23     modifier onlyOwner() {
24         require(msg.sender == owner);
25         _;
26     }
27
28     constructor() {
29         owner = msg.sender;

30         users[owner].tokens = 100;
31     }
32
33     function double(uint _value) public pure returns (uint) {
34         return _value * 2;
35     }
36
37     function register(string memory _name) public {
38         users[msg.sender].name = _name;
39     }
40
41     // Transferencia interna
42     function _transferTokens(address _receiver, uint256 _amount) private {
43         require(users[owner].tokens >= _amount, "No quedan suficientes tokens");
44
45         users[owner].tokens -= _amount;
46         users[_receiver].tokens += _amount;
48
49     function giveToken(address _receiver, uint256 _amount) public onlyOwner {
50         _transferTokens(_receiver, _amount);
51
52         emit TokensGifted(_receiver, _amount);
53     }
55     // comprar tokens con Ether

56     function buyTokens(uint256 _amount) public payable {
57         require(msg.value >= _amount * TOKEN_PRICE, "Ether insuficiente");
58
59         require(users[owner].tokens >= _amount, "No quedan suficientes tokens");
60
66     // Lee el estado, pero no lo modifica
67     function getContractBalance() public view returns (uint256) {
68         return address(this).balance;
69     }
70 }
```

4. E3 — desarrollo y evidencias

Documentación: E3/README.md

Ejercio 3 — MetaMask y red de pruebas Ethereum Sepolia

> **Asignatura:** Blockchain

> **Fecha:** 26 de octubre de 2026

> **Entorno:** MetaMask (extensión de navegador), Ethereum Sepolia, Google Cloud Web3 Faucet y Sepolia Etherscan.

1. Objetivos

Esta práctica tiene como finalidad aprender a configurar una cartera de Ethereum con **MetaMask**, habilitar una red de pruebas, solicitar ETH de prueba mediante un _faucet_, comprobar las operaciones en un explorador de bloques y efectuar una transferencia entre dos direcciones.

Importante: Sepolia es una red de pruebas. Su ETH no tiene valor económico real y no debe confundirse con el ETH de Ethereum Mainnet.

2. Preparación de MetaMask

Paso 1. Acceder al asistente de instalación

Se abre la extensión de MetaMask, que ofrece crear una cartera nueva o importar una existente. Para una práctica se puede utilizar una cartera de pruebas independiente.

!01 - Bienvenida a MetaMask

Paso 2. Finalizar la configuración inicial

Documentación: E3/Readme.md

Ejercio 3 — MetaMask y red de pruebas Ethereum Sepolia

> **Asignatura:** Blockchain

> **Fecha:** 26 de octubre de 2026

> **Entorno:** MetaMask (extensión de navegador), Ethereum Sepolia, Google Cloud Web3 Faucet y Sepolia Etherscan.

1. Objetivos

Esta práctica tiene como finalidad aprender a configurar una cartera de Ethereum con **MetaMask**, habilitar una red de pruebas, solicitar ETH de prueba mediante un _faucet_, comprobar las operaciones en un explorador de bloques y efectuar una transferencia entre dos direcciones.

Importante: Sepolia es una red de pruebas. Su ETH no tiene valor económico real y no debe confundirse con el ETH de Ethereum Mainnet.

2. Preparación de MetaMask

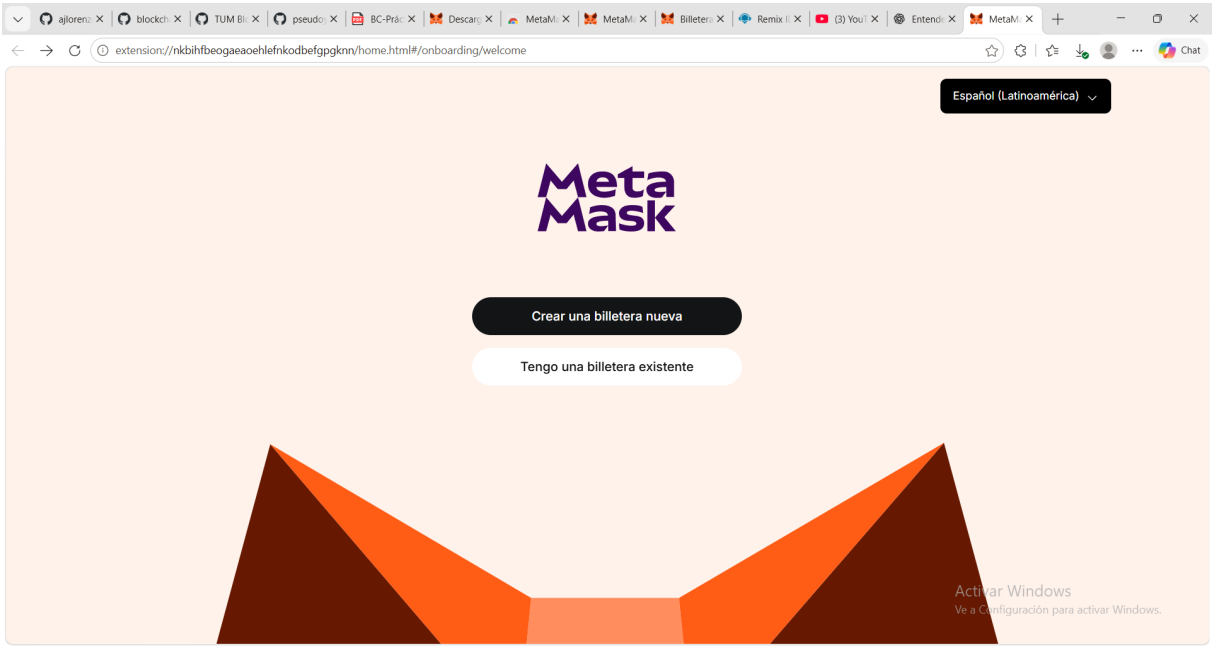
Paso 1. Acceder al asistente de instalación

Se abre la extensión de MetaMask, que ofrece crear una cartera nueva o importar una existente. Para una práctica se puede utilizar una cartera de pruebas independiente.

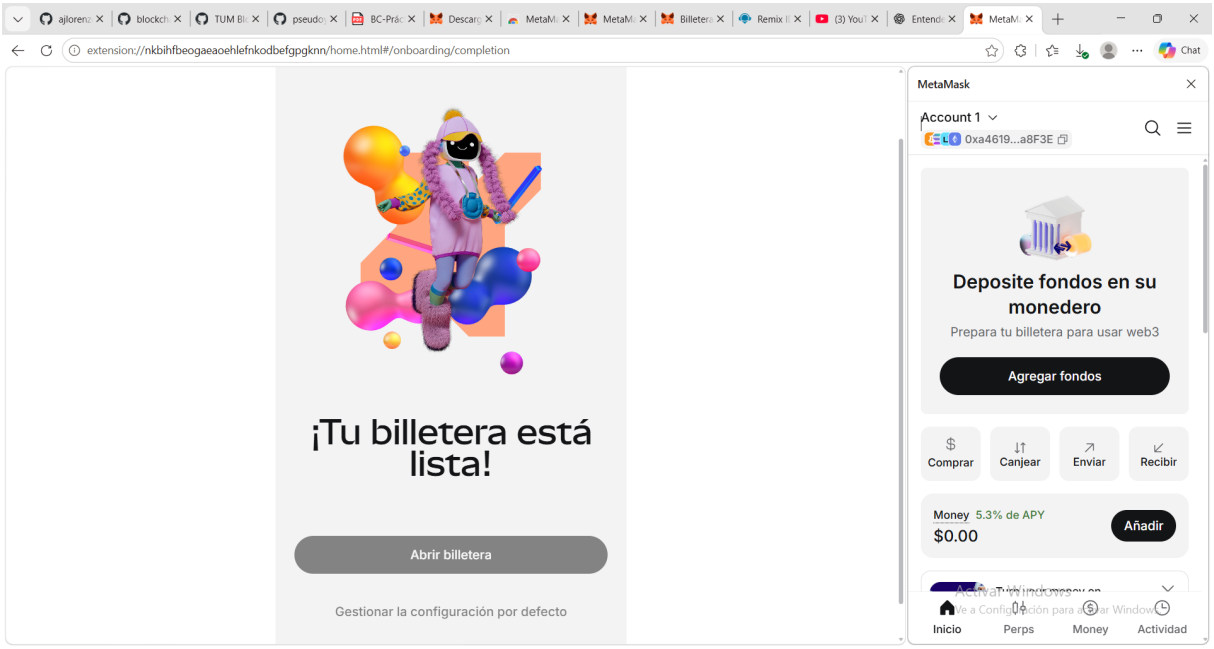
!01 - Bienvenida a MetaMask

Paso 2. Finalizar la configuración inicial

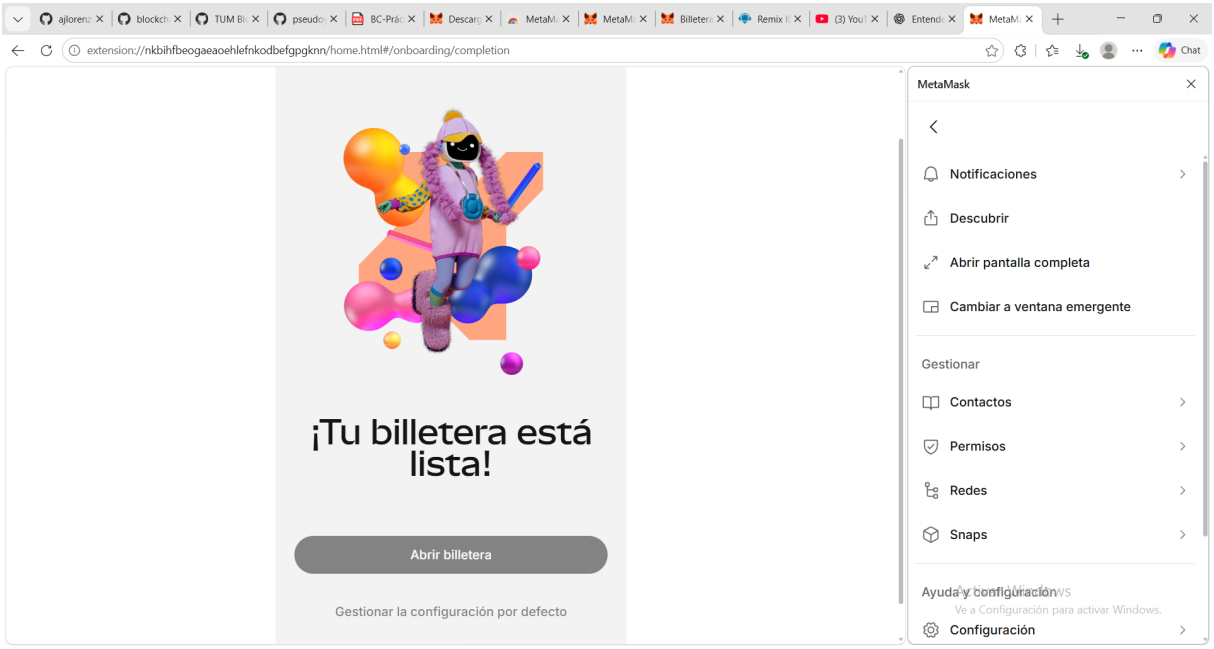
01.png


extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#/onboarding/welcome
Español (Latinoamérica)
Meta Mask
Crear una billetera nueva
Tengo una billetera existente
Activar Windows

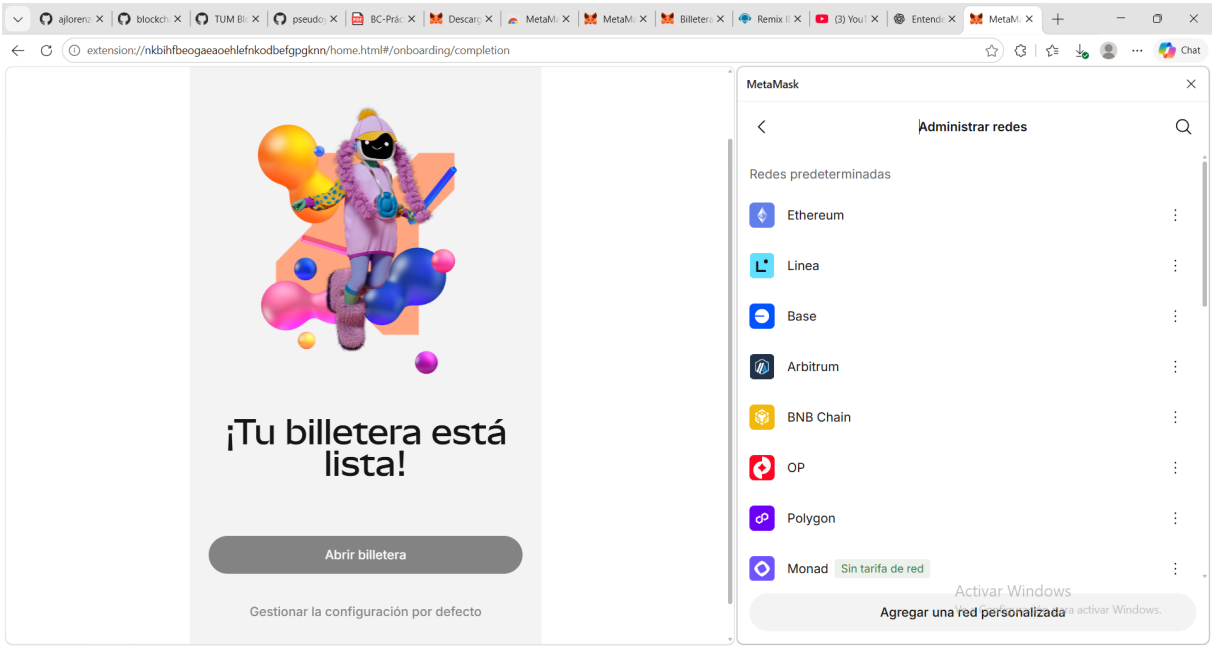
02.png


extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#/onboarding/completion
¡Tu billetera está lista!
Abrir billetera
Gestionar la configuración por defecto
MetaMask
Account 1
0xa4619...a8F3E
Deposite fondos en su monedero
Prepara tu billetera para usar web3
Agregar fondos
Comprar
Canjear
Enviar
Recibir
Money 5.3% de APY
$0.00
Añadir
Inicio
Perps
Money
Actividad

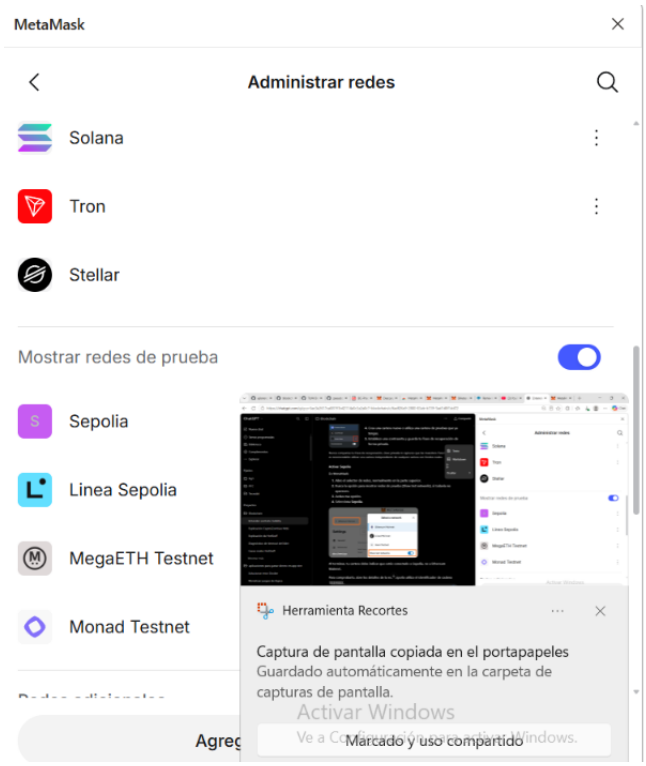
03.png


extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#/onboarding/completion
¡Tu billetera está lista!
Abrir billetera
Gestionar la configuración por defecto
MetaMask
Notificaciones
Descubrir
Abrir pantalla completa
Cambiar a ventana emergente
Gestionar
Contactos
Permisos
Redes
Snaps
Ayuda y configuración
Configuración

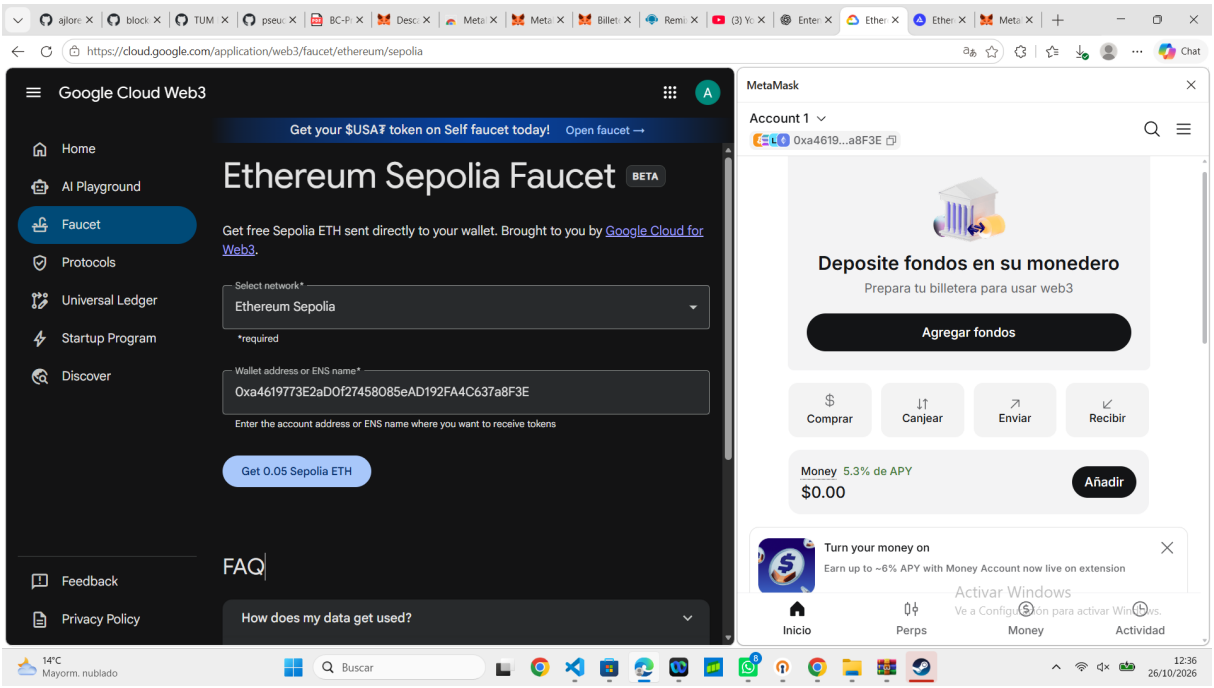
04.png


extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#/onboarding/completion
¡Tu billetera está lista!
Abrir billetera
Gestionar la configuración por defecto
MetaMask
Administrar redes
Redes predeterminadas
Ethereum
Linea
Base
Arbitrum
BNB Chain
OP
Polygon
Monad
Sin tarifa de red
Agregar una red personalizada

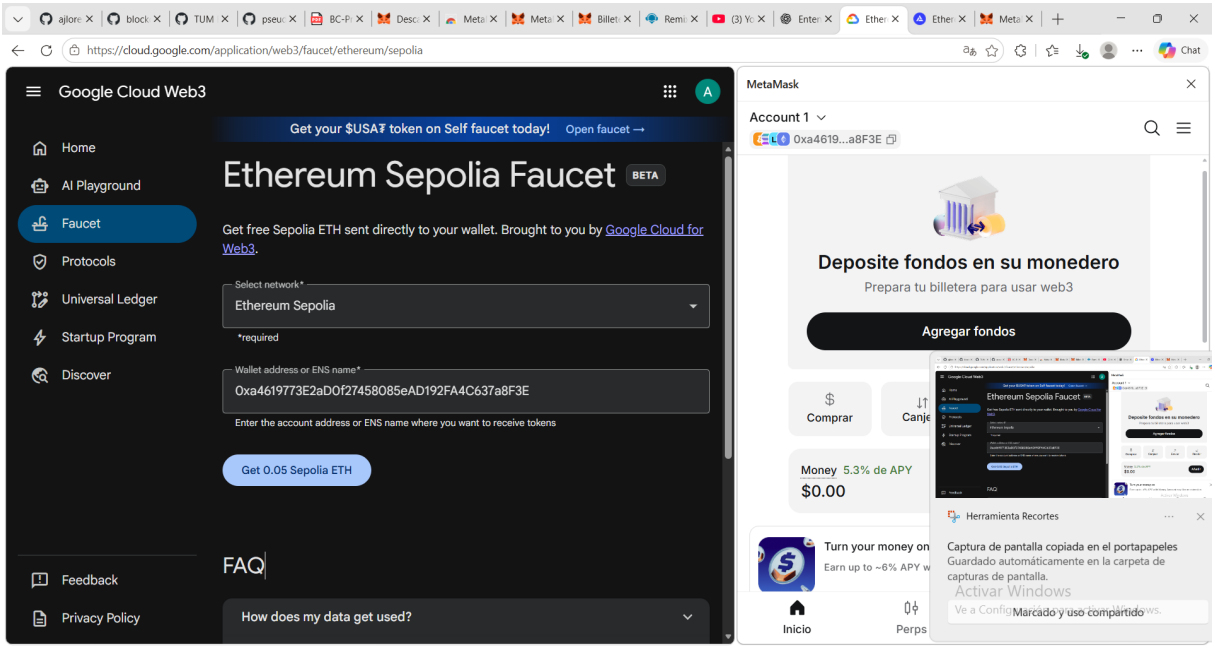
06.png


MetaMask
Administrar redes
Solana
Tron
Stellar
Mostrar redes de prueba
Sepolia
Linea Sepolia
MegaETH Testnet
Monad Testnet
Herramienta Recortes
Captura de pantalla copiada en el portapapeles
Guardado automáticamente en la carpeta de capturas de pantalla.
Activar Windows
Marcado y uso compartido

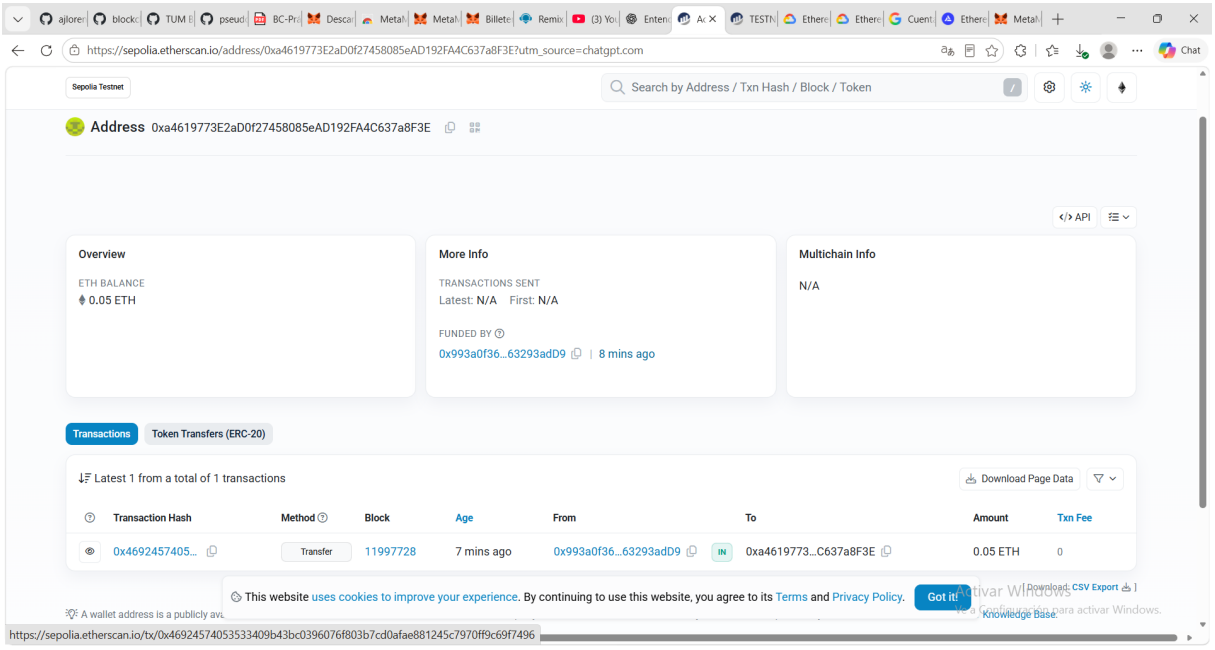
07.png


https://cloud.google.com/application/web3/faucet/ethereum/sepolia
Google Cloud Web3
Get your $USA₮ token on Self faucet today! Open faucet →
Home
AI Playground
Faucet
Protocols
Universal Ledger
Startup Program
Discover
Feedback
Privacy Policy
Ethereum Sepolia Faucet
BETA
Get free Sepolia ETH sent directly to your wallet. Brought to you by Google Cloud for Web3.
Select network*
Ethereum Sepolia
*required
Wallet address or ENS name*
0xa4619773E2aD0f27458085eAD192FA4C637a8F3E
Enter the account address or ENS name where you want to receive tokens
Get 0.05 Sepolia ETH
FAQ
How does my data get used?
MetaMask
Account 1
0xa4619...a8F3E
Deposite fondos en su monedero
Prepara tu billetera para usar web3
Agregar fondos
Comprar
Canjear
Enviar
Recibir
Money 5.3% de APY
$0.00
Añadir
Turn your money on
Earn up to ~6% APY with Money Account now live on extension
Activar Windows
Inicio
Perps
Money
Actividad
Buscar
26/10/2026

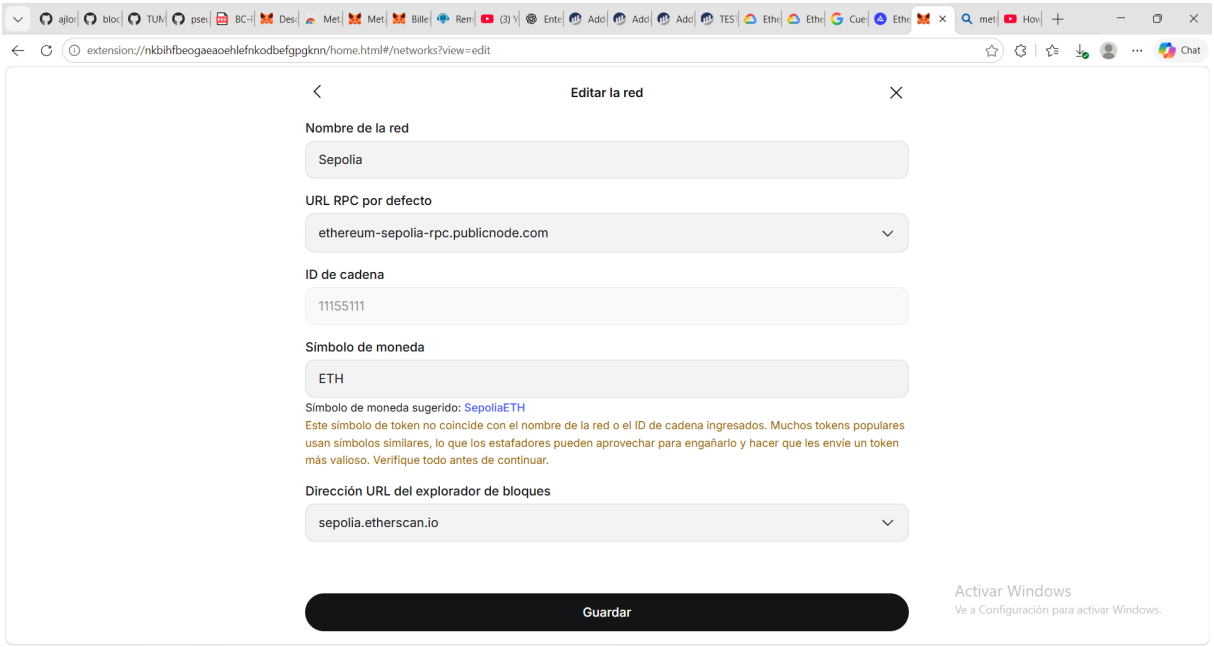
08.png


https://cloud.google.com/application/web3/faucet/ethereum/sepolia
Google Cloud Web3
Get your $USA₮ token on Self faucet today! Open faucet →
Home
AI Playground
Faucet
Protocols
Universal Ledger
Startup Program
Discover
Feedback
Privacy Policy
Ethereum Sepolia Faucet
BETA
Get free Sepolia ETH sent directly to your wallet. Brought to you by Google Cloud for Web3.
Select network*
Ethereum Sepolia
*required
Wallet address or ENS name*
0xa4619773E2aD0f27458085eAD192FA4C637a8F3E
Enter the account address or ENS name where you want to receive tokens
Get 0.05 Sepolia ETH
FAQ
How does my data get used?
MetaMask
Account 1
0xa4619...a8F3E
Deposite fondos en su monedero
Prepara tu billetera para usar web3
Agregar fondos
Comprar
Money 5.3% de APY
$0.00
Turn your money on
Earn up to ~6% APY
Inicio
Herramienta Recortes
Captura de pantalla copiada en el portapapeles
Guardado automáticamente en la carpeta de capturas de pantalla.
Marcado y uso compartido
Activar Windows

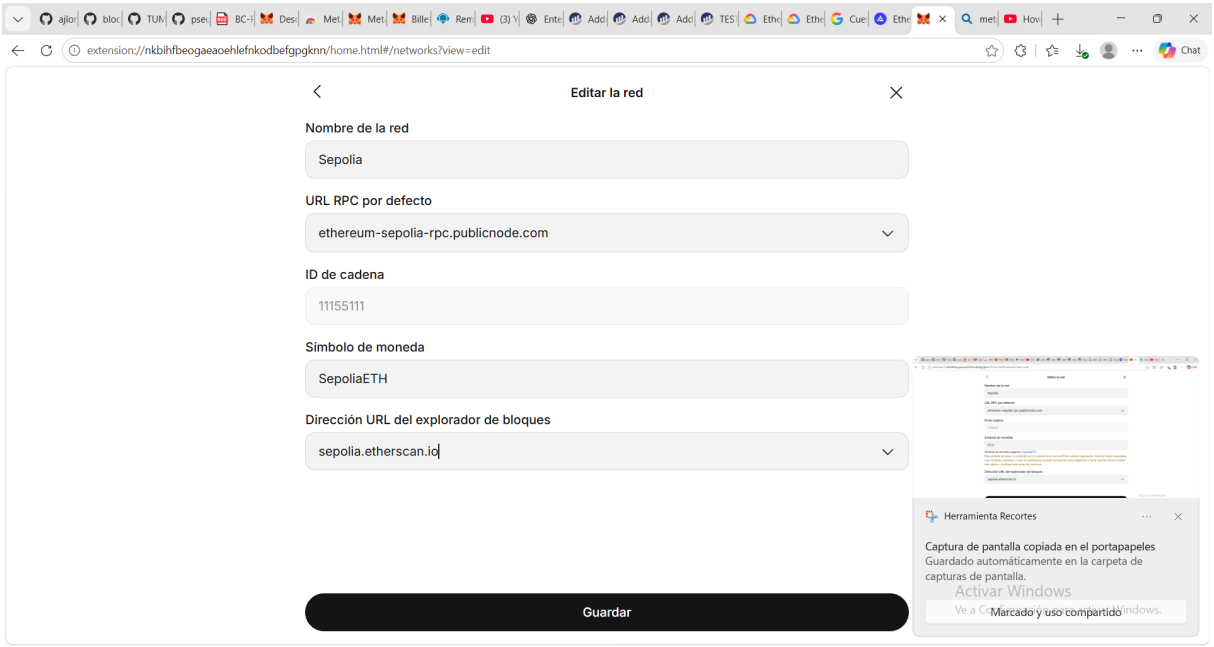
09.png


https://sepolia.etherscan.io/address/0xa4619773E2aD0f27458085eAD192FA4C637a8F3E?utm_source=chatgpt.com
Sepolia Testnet
Search by Address / Txn Hash / Block / Token
Address 0xa4619773E2aD0f27458085eAD192FA4C637a8F3E
API
Overview
ETH BALANCE
0.05 ETH
More Info
TRANSACTIONS SENT
Latest: N/A First: N/A
FUNDED BY
0x993a0f36...63293adD9 | 8 mins ago
Multichain Info
N/A
Transactions
Token Transfers (ERC-20)
Latest 1 from a total of 1 transactions
Download Page Data
Transaction Hash
Method
Block
Age
From
To
Amount
Txn Fee
0x4692457405...
Transfer
11997728
7 mins ago
0x993a0f36...63293adD9
IN
0xa4619773...C637a8F3E
0.05 ETH
0
This website uses cookies to improve your experience. By continuing to use this website, you agree to its Terms and Privacy Policy.
Got it!
Knowledge Base
https://sepolia.etherscan.io/tx/0x4692457405353409b43bc0396076f803b7cd0afae881245c7970ff9c69f7496
Activar Windows

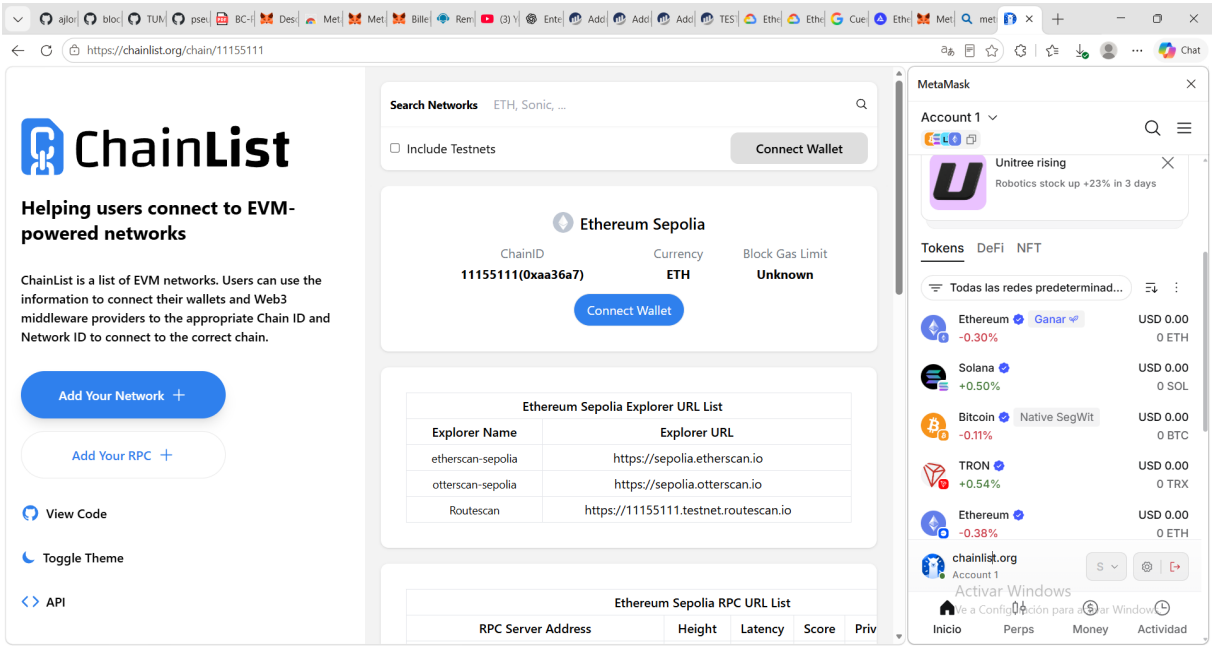
10.png


extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#/networks?view=edit
Editar la red
Nombre de la red
Sepolia
URL RPC por defecto
ethereum-sepolia-rpc.publicnode.com
ID de cadena
11155111
Símbolo de moneda
ETH
Símbolo de moneda sugerido: SepoliaETH
Este símbolo de token no coincide con el nombre de la red o el ID de cadena ingresados. Muchos tokens populares usan símbolos similares, lo que los estafadores pueden aprovechar para engañarlo y hacer que les envíe un token más valioso. Verifique todo antes de continuar.
Dirección URL del explorador de bloques
sepolia.etherscan.io
Guardar
Activar Windows
Ve a Configuración para activar Windows.

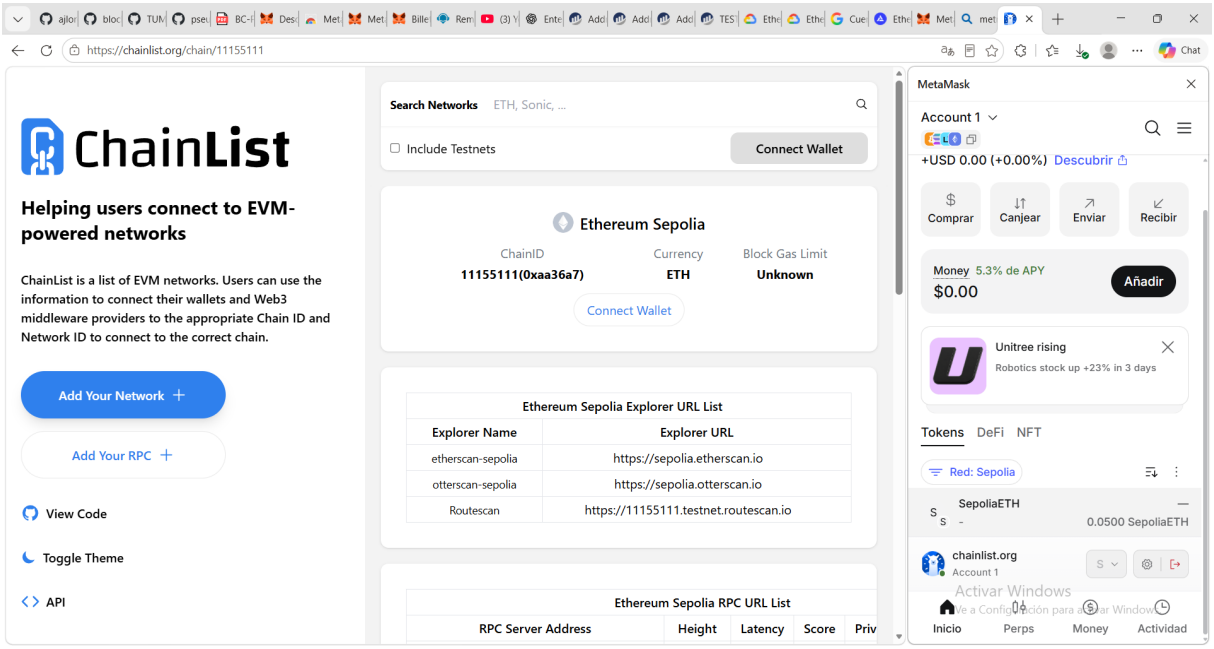
11.png


extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#/networks?view=edit
Editar la red
Nombre de la red
Sepolia
URL RPC por defecto
ethereum-sepolia-rpc.publicnode.com
ID de cadena
11155111
Símbolo de moneda
SepoliaETH
Dirección URL del explorador de bloques
sepolia.etherscan.io
Guardar
Herramienta Recortes
Captura de pantalla copiada en el portapapeles
Guardado automáticamente en la carpeta de capturas de pantalla.
Marcado y uso compartido
Activar Windows

12.png



13.png



5. E4 — desarrollo y evidencias

Documentación: E4/README.md

Ejercicio 4 — Integración de OpenZeppelin en TokenContract

1. Objetivo

El objetivo de este ejercicio es **escoger y probar un contrato de la biblioteca OpenZeppelin Solidity** y guardar el resultado en un archivo `.sol` dentro del repositorio. En lugar de crear un ejemplo independiente, se ha adaptado el contrato `TokenContract` del ejercicio anterior para utilizar los contratos **ERC20** y **Ownable** de OpenZeppelin.

El contrato resultante, `TokenContractOpenZeppelin.sol`, mantiene el registro de usuarios, el regalo de tokens y la compra con ETH, pero delega en OpenZeppelin la contabilidad y transferencia de los tokens y el control de acceso del propietario.

2. Bibliotecas utilizadas

- **ERC20:** implementa el estándar de tokens fungibles. Proporciona `balanceOf`, `transfer`, `approve`, `allowance`, `transferFrom`, `totalSupply` y los eventos estándar.

- **Ownable:** gestiona el propietario mediante `owner()` y restringe operaciones con `onlyOwner`.

La implementación usa la interfaz de **OpenZeppelin Contracts 5.x**, cuyo constructor de `Ownable` recibe la dirección inicial del propietario.

3. Modificaciones respecto al ejercicio anterior

3.1. Propiedad: de `owner` manual a `Ownable`

Antes:

Ahora:

Documentación: E4/Readme.md

Ejercicio 4 — Integración de OpenZeppelin en TokenContract

1. Objetivo

El objetivo de este ejercicio es **escoger y probar un contrato de la biblioteca OpenZeppelin Solidity** y guardar el resultado en un archivo `.sol` dentro del repositorio. En lugar de crear un ejemplo independiente, se ha adaptado el contrato `TokenContract` del ejercicio anterior para utilizar los contratos **ERC20** y **Ownable** de OpenZeppelin.

El contrato resultante, `TokenContractOpenZeppelin.sol`, mantiene el registro de usuarios, el regalo de tokens y la compra con ETH, pero delega en OpenZeppelin la contabilidad y transferencia de los tokens y el control de acceso del propietario.

2. Bibliotecas utilizadas

- **ERC20:** implementa el estándar de tokens fungibles. Proporciona `balanceOf`, `transfer`, `approve`, `allowance`, `transferFrom`, `totalSupply` y los eventos estándar.

- **Ownable:** gestiona el propietario mediante `owner()` y restringe operaciones con `onlyOwner`.

La implementación usa la interfaz de **OpenZeppelin Contracts 5.x**, cuyo constructor de `Ownable` recibe la dirección inicial del propietario.

3. Modificaciones respecto al ejercicio anterior

3.1. Propiedad: de `owner` manual a `Ownable`

Antes:

Ahora:

TokenContractOpenZeppelin.sol

Extracto del archivo E4/TokenContractOpenZeppelin.sol (95 líneas en total).

```
 9
10 contract TokenContractOpenZeppelin is ERC20, Ownable {
11
12     uint256 public constant TOKEN_PRICE = 5 ether;
13
14     event TokensPurchased(address indexed buyer, uint256 amount);
15     event TokensGifted(address indexed receiver, uint256 amount);
16
17     struct Receivers {
18         string name;
19     }
20
21     mapping(address => Receivers) public users;
```

```
22
23      function getUser(address _user) public view returns (string memory name, uint256 tokens) {
24      return (
25          users[_user].name,
26          balanceOf(_user) / 1 ether;
27      );
29
30      constructor() ERC20("Token Practica", "TPR") Ownable(msg.sender)
31      {
32          _mint(msg.sender, 100 * 10 ** decimals());
33      }
34
35      function double(uint256 _value) public pure returns (uint256)
36      {
37          return _value * 2;
38      }
39
40      function register(string memory _name) public {
41          users[msg.sender].name = _name;
42      }
43
44      function _transferTokens(address _receiver, uint256 _amount) private {
45          require(_amount > 0, "Cantidad invalida");
46          require(_receiver != address(0), "Direccion invalida");
47
48          uint256 units = _amount * 10 ** decimals();
55
56      function giveToken( address _receiver, uint256 _amount) public onlyOwner {
57          _transferTokens(_receiver, _amount);
58          emit TokensGifted(_receiver, _amount);
59      }
60
61      function buyTokens(uint256 _amount) public payable {
62          require(_amount > 0, "Cantidad invalida");
63
64          require(
65              msg.value == _amount * TOKEN_PRICE,
72
73      function getContractBalance() public view returns (uint256)
74      {
75          return address(this).balance;
76      }
77
```

6. E5 — PharmaChain Ethereum

Proyecto de trazabilidad de lotes farmacéuticos mediante contratos inteligentes, control de roles y cadena de custodia.

Documentación: E5/PharmaChain-Ethereum/README.md

PharmaChain-Ethereum — Ejercicio 5

Práctica 1 · Tecnologías de Registro Distribuido y Blockchain · Universidade de Vigo · 2026–2027

Prototipo académico de trazabilidad farmacéutica en Ethereum: registra lotes, administra permisos, propone y acepta transferencias de custodia y permite comparar implementaciones por comportamiento y consumo de gas.

> **Alcance:** demostración en una red local Hardhat. No verifica la autenticidad física de medicamentos, no es una solución sanitaria certificada y no debe exponerse a Internet con las cuentas desbloqueadas del laboratorio.

Índice y documentación

- **Este README**: motivación, actores, arquitectura, contratos, decisiones, comparativas lógicas y pruebas.

- **Guía de instalación y demostraciones**: comandos exactos, cuentas, operaciones, auditoría, errores habituales.

- **Benchmarks y optimización**: tabla comparativa de versiones, experimentos V7–V12, gas, metodología y resultados.

1. Problema y justificación

La distribución de medicamentos requiere conocer quién fabrica un lote, quién lo recibe, quién lo custodia y cómo cambia esa responsabilidad. En un registro centralizado, una sola organización administra las operaciones; con un contrato inteligente, varias entidades pueden verificar y ejecutar reglas compartidas. Ethereum aporta estado verificable, eventos de auditoría y autorización ejecutada por la EVM. **No garantiza la veracidad del dato introducido**: la identificación física del lote, las entregas y la temperatura requieren controles externos.

Documentación: E5/PharmaChain-Ethereum/docs/GUIA_USO.md

PharmaChain — Guía de instalación, operación y demostraciones

Volver al README · Benchmarks

1. Requisitos

- Node.js y npm compatibles con las dependencias instaladas.

- Proyecto `PharmaChain-Ethereum/` con `package.json`, `hardhat.config.ts`, `truffle-config.cjs` y contratos.

- Navegador web moderno.

- Puertos locales disponibles: `8545` (RPC), `3100` (laboratorio), `5173` (Vite).

- **No hace falta geth**: el nodo Hardhat proporciona JSON-RPC.

Compatibilidad: Truffle 5 es legado y puede fallar con versiones recientes de Node. La documentación original recomienda probar Node 20 LTS para Truffle si hay incompatibilidades, manteniendo una versión de Node compatible con Hardhat 3 para el resto. No presupongas que una sola versión de Node funciona para ambos sin comprobar las dependencias.

2. Instalación inicial

Documentación: E5/PharmaChain-Ethereum/docs/BENCHMARK.md

PharmaChain-Ethereum — Memoria técnica, benchmark y laboratorio de gas

> **Ejercicio E5 · Ethereum local · Solidity 0.8.34 · Hardhat/Truffle · frontend Vite.** Documento de resultados basado en los CSV disponibles y la documentación aportada. Los números son mediciones registradas, no una nueva ejecución de tests.

1. Objetivo y arquitectura

PharmaChain implementa trazabilidad de lotes farmacéuticos mediante registro, propuesta, aceptación y cancelación de transferencias, desactivación, historial y autorización por actores (administrador, laboratorio, distribuidor, farmacia). Se comparan dos políticas: **roles manuales** (rol exclusivo por cuenta) y **OpenZeppelin AccessControl** (roles múltiples y administración más flexible). Una red Hardhat local (chain ID 31337) permite operar sin costes reales; Truffle se utiliza para migraciones y comparación de herramientas; Web3.js y ethers interactúan con los contratos, mientras que Vite sirve la interfaz y el laboratorio Express present

1.1 Flujo funcional

1. Administrador configura roles. 2. Laboratorio registra lote. 3. Custodio propone transferencia a distribuidor. 4. Destinatario acepta. 5. Distribuidor propone a farmacia y esta acepta. 6. Se registra la custodia y pueden estudiarse cancelación, pausa y desactivación. El contrato rechaza actores no autorizados y saltos de etapa.

1.2 Catálogo del frontend: V0–V7

Aviso: esta numeración de ocho contratos no equivale a V0–V12 del laboratorio de gas. Comparar versiones dentro de la misma familia para atribuir ahorros; la semántica de permisos es distinta entre

familias.

2. GasLab: trece técnicas V0–V12 en dos familias

Los contratos se replican en las familias `Manual` y `OZ`: **2 × 13 = 26**. El documento GasLab EXT describe 156 escenarios previstos de suite principal y 36 de invariantes, pero ese diseño no debe interpretarse automáticamente como 192 tests superados.

Contrato: PharmaChainRolesManuales.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/PharmaChainRolesManuales.sol (72 líneas en total).

```
 6 // Versión didáctica: permisos programados manualmente.
 7 contract PharmaChainRolesManuales is GestionLotesYTransferencias {
 8     address public immutable administrador;
 9     bool public paused;
10 
11     enum Rol {
18 
19     event RolAsignado(address indexed usuario, Rol rol);
20     event Paused(address indexed usuario);
21     event Unpaused(address indexed usuario);
22 
23     constructor() {
24         administrador = msg.sender;
25     }
26 
27     modifier soloAdministrador() {
28         require(msg.sender == administrador, "No eres el administrador");
29         _;
30     }
31 
32     // El administrador asigna un único rol por dirección.
33     function asignarRol(address usuario, Rol rol) external soloAdministrador {
34         require(usuario != address(0), "Direccion invalida");
35         require(usuario != administrador, "El administrador no puede tener rol");

36         roles[usuario] = rol;
37         emit RolAsignado(usuario, rol);
39 
40     function pausar() external soloAdministrador {
41         require(!paused, "Ya pausado");
42         paused = true;
43         emit Paused(msg.sender);
44     }
45 
46     function reanudar() external soloAdministrador {
47         require(paused, "No esta pausado");
48         paused = false;
49         emit Unpaused(msg.sender);
50     }
51 
52     function _esLaboratorio(address usuario) internal view override returns (bool) {
53         return roles[usuario] == Rol.Laboratorio;
54     }
55 
56     function _esAdministrador(address usuario) internal view override returns (bool) {
57         return usuario == administrador;
58     }
59 
60     function _transicionValida(

61         address origen,
62         address destino
63     ) internal view override returns (bool) {
64         return
```

Contrato: PharmaChainRolesManualesOptimized.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/PharmaChainRolesManualesOptimized.sol (72 líneas en total).

```
 6 // Versión didáctica: permisos programados manualmente.
 7 contract PharmaChainRolesManualesOptimized is GestionLotesYTransferenciasOptimized {
 8     address public immutable administrador;
 9     bool public paused;
10 
```

```
11      enum Rol {
18
19      event RolAsignado(address indexed usuario, Rol rol);
20      event Paused(address indexed usuario);
21      event Unpaused(address indexed usuario);
22
23      constructor() {
24          administrador = msg.sender;
25      }
26
27      modifier soloAdministrador() {
28          require(msg.sender == administrador, "No eres el administrador");
29          _;
30      }
31
32      // El administrador asigna un único rol por dirección.
33      function asignarRol(address usuario, Rol rol) external soloAdministrador {
34          require(usuario != address(0), "Direccion invalida");
35          require(usuario != administrador, "El administrador no puede tener rol");

36          roles[usuario] = rol;
37          emit RolAsignado(usuario, rol);
39
40      function pausar() external soloAdministrador {
41          require(!paused, "Ya pausado");
42          paused = true;
43          emit Paused(msg.sender);
44      }
45
46      function reanudar() external soloAdministrador {
47          require(paused, "No esta pausado");
48          paused = false;
49          emit Unpaused(msg.sender);
50      }
51
52      function _esLaboratorio(address usuario) internal view override returns (bool) {
53          return roles[usuario] == Rol.Laboratorio;
54      }
55
56      function _esAdministrador(address usuario) internal view override returns (bool) {
57          return usuario == administrador;
58      }
59
60      function _transicionValida(

61          address origen,
62          address destino
63      ) internal view override returns (bool) {
64          return
```

Contrato: PharmaChainRolesManualesOptimized1.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/PharmaChainRolesManualesOptimized1.sol (79 líneas en total).

```
 6  // Versión didáctica: permisos programados manualmente.
 7  contract PharmaChainRolesManualesOptimized1 is GestionLotesYTransferenciasOptimized1 {
 8      error ContratoPausado();
 9      error DireccionInvalida();
10      error AdministradorConRol();
11      error YaPausado();
12      error NoPausado();
13      error NoAdministrador();
14
15      address public immutable administrador;
16      bool public paused;
17
25
26      event RolAsignado(address indexed usuario, Rol rol);
27      event Paused(address indexed usuario);
28      event Unpaused(address indexed usuario);
29
30      constructor() {
31          administrador = msg.sender;
32      }
33
34      modifier soloAdministrador() {
35          if (!(msg.sender == administrador)) revert NoAdministrador();
36          _;

37      }
```

```
38
```

Contrato: PharmaChainRolesManualesOptimizednoID.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/PharmaChainRolesManualesOptimizednoID.sol (60 líneas en total).

```
 5
 6 contract PharmaChainRolesManualesOptimizednoID is GestionLotesYTransferenciasOptimizednoID {
 7     address public immutable administrador;
 8     bool public paused;
 9
10     enum Rol { Ninguno, Laboratorio, Distribuidor, Farmacia }
12
13     event RolAsignado(address indexed usuario, Rol rol);
14     event Paused(address indexed usuario);
15     event Unpaused(address indexed usuario);
16
17     constructor() { administrador = msg.sender; }
18
19     modifier soloAdministrador() {
20         require(msg.sender == administrador, "No eres el administrador");
21         _;
22     }
23
24     function asignarRol(address usuario, Rol rol) external soloAdministrador {
25         require(usuario != address(0), "Direccion invalida");
26         require(usuario != administrador, "El administrador no puede tener rol");
27         roles[usuario] = rol;
28         emit RolAsignado(usuario, rol);
30
31     function pausar() external soloAdministrador {
32         require(!paused, "Ya pausado");
33         paused = true;
34         emit Paused(msg.sender);
35     }
36
37     function reanudar() external soloAdministrador {
38         require(paused, "No esta pausado");
39         paused = false;
40         emit Unpaused(msg.sender);
41     }
42
43     function _esLaboratorio(address usuario) internal view override returns (bool) {
44         return roles[usuario] == Rol.Laboratorio;
45     }
46
47     function _esAdministrador(address usuario) internal view override returns (bool) {
48         return usuario == administrador;
49     }
50
51     function _transicionValida(address origen, address destino) internal view override returns
52         Rol rolOrigen = roles[origen];
53         return (rolOrigen == Rol.Laboratorio && roles[destino] == Rol.Distribuidor)
54             || (rolOrigen == Rol.Distribuidor && roles[destino] == Rol.Farmacia);
55     }
```

Contrato: PharmaChainRolesOpenZeppelin.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/PharmaChainRolesOpenZeppelin.sol (49 líneas en total).

```
 8 // Versión con controles de seguridad de OpenZeppelin.
 9 contract PharmaChainRolesOpenZeppelin is GestionLotesYTransferencias, AccessControl, Pausable
10     bytes32 public constant LABORATORIO_ROLE = keccak256("LABORATORIO_ROLE");
11     bytes32 public constant DISTRIBUIDOR_ROLE = keccak256("DISTRIBUIDOR_ROLE");
12     bytes32 public constant FARMACIA_ROLE = keccak256("FARMACIA_ROLE");
13
16
17     constructor() {
18         administrador = msg.sender;
19         _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
20     }
21
22     function pausar() external onlyRole(DEFAULT_ADMIN_ROLE) {
23         _pause();
24     }
25     function reanudar() external onlyRole(DEFAULT_ADMIN_ROLE) {
```

```
        _unpause();
    }

    function _esLaboratorio(address usuario) internal view override returns (bool) {
        return hasRole(LABORATORIO_ROLE, usuario);
    }

    function _esAdministrador(address usuario) internal view override returns (bool) {
        return hasRole(DEFAULT_ADMIN_ROLE, usuario);
    }

    function _transicionValida(
        address origen,
        address destino
    ) internal view override returns (bool) {
        return

    function _comprobarActivo() internal view override {
        _requireNotPaused();
    }
}
```

Contrato: PharmaChainRolesOpenZeppelinOptimized.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/PharmaChainRolesOpenZeppelinOptimized.sol (53 líneas en total).

```
// Versión con controles de seguridad de OpenZeppelin.
contract PharmaChainRolesOpenZeppelinOptimized is
    GestionLotesYTransferenciasOptimized,
    AccessControl,
    Pausable
{

    constructor() {
        administrador = msg.sender;
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }

    function pausar() external onlyRole(DEFAULT_ADMIN_ROLE) {
        _pause();
    }
    function reanudar() external onlyRole(DEFAULT_ADMIN_ROLE) {
        _unpause();
    }

    function _esLaboratorio(address usuario) internal view override returns (bool) {
        return hasRole(LABORATORIO_ROLE, usuario);
    }

    function _esAdministrador(address usuario) internal view override returns (bool) {
        return hasRole(DEFAULT_ADMIN_ROLE, usuario);
    }

    function _transicionValida(
        address origen,
        address destino
    ) internal view override returns (bool) {
        return

    function _comprobarActivo() internal view override {
        _requireNotPaused();
    }
}
```

Contrato: PharmaChainRolesOpenZeppelinOptimized1.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/PharmaChainRolesOpenZeppelinOptimized1.sol (53 líneas en total).

```
// Versión con controles de seguridad de OpenZeppelin.
contract PharmaChainRolesOpenZeppelinOptimized1 is
    GestionLotesYTransferenciasOptimized1,
    AccessControl,
    Pausable
{

    constructor() {
```

```
22         administrador = msg.sender;
23         _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
24     }
25
26     function pausar() external onlyRole(DEFAULT_ADMIN_ROLE) {
27         _pause();
28     }
29     function reanudar() external onlyRole(DEFAULT_ADMIN_ROLE) {
30         _unpause();
31     }
32
33     function _esLaboratorio(address usuario) internal view override returns (bool) {
34         return hasRole(LABORATORIO_ROLE, usuario);
35     }
36
37     function _esAdministrador(address usuario) internal view override returns (bool) {

38         return hasRole(DEFAULT_ADMIN_ROLE, usuario);
39     }
40
41     function _transicionValida(
42         address origen,
43         address destino
44     ) internal view override returns (bool) {
45         if (hasRole(LABORATORIO_ROLE, origen)) return hasRole(DISTRIBUIDOR_ROLE, destino);
49
50     function _comprobarActivo() internal view override {
51         _requireNotPaused();
52     }
53 }
```

Contrato: PharmaChainRolesOpenZeppelinOptimizednoID.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/PharmaChainRolesOpenZeppelinOptimizednoID.sol (36 líneas en total).

```
 7
 8 contract PharmaChainRolesOpenZeppelinOptimizednoID is GestionLotesYTransferenciasOptimizednoID
 9     bytes32 public constant LABORATORIO_ROLE = keccak256("LABORATORIO_ROLE");
10     bytes32 public constant DISTRIBUIDOR_ROLE = keccak256("DISTRIBUIDOR_ROLE");
11     bytes32 public constant FARMACIA_ROLE = keccak256("FARMACIA_ROLE");
12     address public immutable administrador;
13
14     constructor() {
15         administrador = msg.sender;
16         _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
17     }
18
19     function pausar() external onlyRole(DEFAULT_ADMIN_ROLE) { _pause(); }
20     function reanudar() external onlyRole(DEFAULT_ADMIN_ROLE) { _unpause(); }
21
22     function _esLaboratorio(address usuario) internal view override returns (bool) {
23         return hasRole(LABORATORIO_ROLE, usuario);
24     }
25
26     function _esAdministrador(address usuario) internal view override returns (bool) {
27         return hasRole(DEFAULT_ADMIN_ROLE, usuario);
28     }
29
30     function _transicionValida(address origen, address destino) internal view override returns

31         return (hasRole(LABORATORIO_ROLE, origen) && hasRole(DISTRIBUIDOR_ROLE, destino))
32             || (hasRole(DISTRIBUIDOR_ROLE, origen) && hasRole(FARMACIA_ROLE, destino));
33     }
34
35     function _comprobarActivo() internal view override { _requireNotPaused(); }
36 }
```

7. E5 — Laboratorio de optimización GasLab

La numeración V0–V12 del laboratorio no coincide con las ocho variantes V0–V7 del selector principal. Las técnicas son hipótesis que deben contrastarse con el código, los tests y los CSV.

Versión	Técnica evaluada
V0Base	Línea base de referencia

Versión	Técnica evaluada
V1Roles	Gestión y comprobación de roles
V2Errores	Errores personalizados (custom errors)
V3Storage	Optimización de accesos a storage
V4Unchecked	Operaciones unchecked cuando es seguro
V5Combinada	Combinación de optimizaciones anteriores
V6SinId	Eliminar el ID redundante de la estructura
V7LotePacked	Empaquetado de campos del lote
V8HistorialPacked	Empaquetado del historial
V9TransferenciaCompacta	Transferencia compacta y origen derivado
V10PackingCombinado	Combinación de optimizaciones de packing
V11SoloEventos	Historial mediante eventos; cambia la API de consulta
V12EventosCompacta	Eventos y empaquetado combinados

Archivos Solidity de GasLab encontrados: 47.

contracts/gas-lab/GasLabBase.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GasLabBase.sol (72 líneas en total).

```
 6 // Versión didáctica: permisos programados manualmente.
 7 contract GasLabBase is GestionLotesYTransferencias {
 8     address public immutable administrador;
 9     bool public paused;
10 
11     enum Rol {
18 
19     event RolAsignado(address indexed usuario, Rol rol);
20     event Paused(address indexed usuario);
21     event Unpaused(address indexed usuario);
22 
23     constructor() {
24         administrador = msg.sender;
25     }
26 
27     modifier soloAdministrador() {
28         require(msg.sender == administrador, "No eres el administrador");
29         _;
30     }
31 
32     // El administrador asigna un único rol por dirección.
33     function asignarRol(address usuario, Rol rol) external soloAdministrador {
34         require(usuario != address(0), "Direccion invalida");
35         require(usuario != administrador, "El administrador no puede tener rol");
```

contracts/gas-lab/GasLabCache.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GasLabCache.sol (73 líneas en total).

```
 6 // Versión didáctica: permisos programados manualmente.
 7 contract GasLabCache is GestionLotesYTransferencias {
 8     address public immutable administrador;
 9     bool public paused;
10 
11     enum Rol {
18 
19     event RolAsignado(address indexed usuario, Rol rol);
20     event Paused(address indexed usuario);
21     event Unpaused(address indexed usuario);
22 
23     constructor() {
24         administrador = msg.sender;
25     }
26 
27     modifier soloAdministrador() {
28         require(msg.sender == administrador, "No eres el administrador");
```

```
29         _;
30     }
31
32     // El administrador asigna un único rol por dirección.
33     function asignarRol(address usuario, Rol rol) external soloAdministrador {
34         require(usuario != address(0), "Direccion invalida");
35         require(usuario != administrador, "El administrador no puede tener rol");
```

contracts/gas-lab/GasLabCacheErrors.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GasLabCacheErrors.sol (79 líneas en total).

```
 6 // Versión didáctica: permisos programados manualmente.
 7 contract GasLabCacheErrors is GestionLotesYTransferencias {
 8     error NoAdministrador();
 9     error DireccionInvalida();
10     error AdministradorConRol();
11     error YaPausado();
12     error NoEstaPausado();
13     error ContratoPausado();
14     address public immutable administrador;
15     bool public paused;
16
17     enum Rol {
24
25     event RolAsignado(address indexed usuario, Rol rol);
26     event Paused(address indexed usuario);
27     event Unpaused(address indexed usuario);
28
29     constructor() {
30         administrador = msg.sender;
31     }
32
33     modifier soloAdministrador() {
34         if (msg.sender != administrador) revert NoAdministrador();
35         _;
```

contracts/gas-lab/GasLabErrors.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GasLabErrors.sol (78 líneas en total).

```
 6 // Versión didáctica: permisos programados manualmente.
 7 contract GasLabErrors is GestionLotesYTransferencias {
 8     error NoAdministrador();
 9     error DireccionInvalida();
10     error AdministradorConRol();
11     error YaPausado();
12     error NoEstaPausado();
13     error ContratoPausado();
14     address public immutable administrador;
15     bool public paused;
16
17     enum Rol {
24
25     event RolAsignado(address indexed usuario, Rol rol);
26     event Paused(address indexed usuario);
27     event Unpaused(address indexed usuario);
28
29     constructor() {
30         administrador = msg.sender;
31     }
32
33     modifier soloAdministrador() {
34         if (msg.sender != administrador) revert NoAdministrador();
35         _;
```

contracts/gas-lab/GestionBase.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionBase.sol (188 líneas en total).

```
 4 // Lógica común: lotes y cadena de custodia.
 5 abstract contract GestionBase {
 6     uint256 public totalLotes;
 7
 8     struct Lote {
 9         uint256 id;
10         string nombre;
11         string fabricante;
```

```
12          address propietario;
16
17      struct Transferencia {
18          address origen;
19          address destino;
20          bool pendiente;
21      }
25
26      event LoteRegistrado(uint256 indexed id, string nombre, address indexed propietario);
27      event TransferenciaPropuesta(
28          uint256 indexed id,
29          address indexed origen,
30          address indexed destino
31      );
32      event TransferenciaAceptada(
33          uint256 indexed id,
```

contracts/gas-lab/GestionGasOptimizada.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionGasOptimizada.sol (211 líneas en total).

```
 4 // Lógica común: lotes y cadena de custodia.
 5 abstract contract GestionGasOptimizada {
 6     error LoteInexistente(uint256 id);
 7     error NoLaboratorio();
 8     error NombreVacio();
 9     error FabricanteVacio();
10     error LoteInactivo();
11     error NoCustodio();
12     error TransferenciaPendiente();
13     error DestinoInvalido();
14     error TransicionInvalida();
15     error SinTransferencia();
16     error NoDestinatario();
17     error CustodioInconsistente();
18     error NoAutorizado();
19     error LoteYaInactivo();
20     error MovimientoInexistente(uint256 indice);
```

contracts/gas-lab/GestionV0Base.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV0Base.sol (188 líneas en total).

```
 4 // Lógica común: lotes y cadena de custodia.
 5 abstract contract GestionV0Base {
 6     uint256 public totalLotes;
 7
 8     struct Lote {
 9         uint256 id;
10         string nombre;
11         string fabricante;
12         address propietario;
16
17     struct Transferencia {
18         address origen;
19         address destino;
20         bool pendiente;
21     }
25
26     event LoteRegistrado(uint256 indexed id, string nombre, address indexed propietario);
27     event TransferenciaPropuesta(
28         uint256 indexed id,
29         address indexed origen,
30         address indexed destino
31     );
32     event TransferenciaAceptada(
33         uint256 indexed id,
```

contracts/gas-lab/GestionV10PackingCombinado.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV10PackingCombinado.sol (209 líneas en total).

```
 4 // Lógica común: lotes y cadena de custodia.
 5 abstract contract GestionV10PackingCombinado {
 6     error NombreVacio();
 7     error TransicionInvalida();
 8     error NoDestinatario();
```

```
    error NoAutorizado();
    error FabricanteVacio();
    error TransferenciaPendiente();
    error SinTransferencia();
    error YaInactivo();
    error DestinoInvalido();
    error NoLaboratorio();
    error MovimientoInexistente();
    error NoCustodio();
    error LoteInactivo();
    error LoteInexistente();
    error CustodioInconsistente();
```

contracts/gas-lab/GestionV11SoloEventos.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV11SoloEventos.sol (189 líneas en total).

```
// Lógica común: lotes y cadena de custodia.
abstract contract GestionV11SoloEventos {
    error NombreVacio();
    error TransicionInvalida();
    error NoDestinatario();
    error NoAutorizado();
    error FabricanteVacio();
    error TransferenciaPendiente();
    error SinTransferencia();
    error YaInactivo();
    error DestinoInvalido();
    error NoLaboratorio();
    error MovimientoInexistente();
    error NoCustodio();
    error LoteInactivo();
    error LoteInexistente();
    error CustodioInconsistente();
```

contracts/gas-lab/GestionV12EventosCompacta.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV12EventosCompacta.sol (191 líneas en total).

```
// Lógica común: lotes y cadena de custodia.
abstract contract GestionV12EventosCompacta {
    error NombreVacio();
    error TransicionInvalida();
    error NoDestinatario();
    error NoAutorizado();
    error FabricanteVacio();
    error TransferenciaPendiente();
    error SinTransferencia();
    error YaInactivo();
    error DestinoInvalido();
    error NoLaboratorio();
    error MovimientoInexistente();
    error NoCustodio();
    error LoteInactivo();
    error LoteInexistente();
    error CustodioInconsistente();
```

contracts/gas-lab/GestionV1Roles.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV1Roles.sol (188 líneas en total).

```
// Lógica común: lotes y cadena de custodia.
abstract contract GestionV1Roles {
    uint256 public totalLotes;

    struct Lote {
        uint256 id;
        string nombre;
        string fabricante;
        address propietario;

    struct Transferencia {
        address origen;
        address destino;
        bool pendiente;
    }

```

```
    event LoteRegistrado(uint256 indexed id, string nombre, address indexed propietario);
    event TransferenciaPropuesta(
        uint256 indexed id,
        address indexed origen,
        address indexed destino
    );
    event TransferenciaAceptada(
        uint256 indexed id,
```

contracts/gas-lab/GestionV2Errores.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV2Errores.sol (204 líneas en total).

```
// Lógica común: lotes y cadena de custodia.
abstract contract GestionV2Errores {
    error NombreVacio();
    error TransicionInvalida();
    error NoDestinatario();
    error NoAutorizado();
    error FabricanteVacio();
    error TransferenciaPendiente();
    error SinTransferencia();
    error YaInactivo();
    error DestinoInvalido();
    error NoLaboratorio();
    error MovimientoInexistente();
    error NoCustodio();
    error LoteInactivo();
    error LoteInexistente();
    error CustodioInconsistente();
```

contracts/gas-lab/GestionV3Storage.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV3Storage.sol (193 líneas en total).

```
// Lógica común: lotes y cadena de custodia.
abstract contract GestionV3Storage {
    uint256 public totalLotes;

    struct Lote {
        uint256 id;
        string nombre;
        string fabricante;
        address propietario;

    struct Transferencia {
        address origen;
        address destino;
        bool pendiente;
    }

    event LoteRegistrado(uint256 indexed id, string nombre, address indexed propietario);
    event TransferenciaPropuesta(
        uint256 indexed id,
        address indexed origen,
        address indexed destino
    );
    event TransferenciaAceptada(
        uint256 indexed id,
```

contracts/gas-lab/GestionV4Unchecked.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV4Unchecked.sol (188 líneas en total).

```
// Lógica común: lotes y cadena de custodia.
abstract contract GestionV4Unchecked {
    uint256 public totalLotes;

    struct Lote {
        uint256 id;
        string nombre;
        string fabricante;
        address propietario;

    struct Transferencia {
        address origen;
        address destino;
```

```
20          bool pendiente;
21      }
25
26      event LoteRegistrado(uint256 indexed id, string nombre, address indexed propietario);
27      event TransferenciaPropuesta(
28          uint256 indexed id,
29          address indexed origen,
30          address indexed destino
31      );
32      event TransferenciaAceptada(
33          uint256 indexed id,
```

contracts/gas-lab/GestionV5Combinada.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV5Combinada.sol (209 líneas en total).

```
 4 // Lógica común: lotes y cadena de custodia.
 5 abstract contract GestionV5Combinada {
 6     error NombreVacio();
 7     error TransicionInvalida();
 8     error NoDestinatario();
 9     error NoAutorizado();
10     error FabricanteVacio();
11     error TransferenciaPendiente();
12     error SinTransferencia();
13     error YaInactivo();
14     error DestinoInvalido();
15     error NoLaboratorio();
16     error MovimientoInexistente();
17     error NoCustodio();
18     error LoteInactivo();
19     error LoteInexistente();
20     error CustodioInconsistente();
```

contracts/gas-lab/GestionV6SinId.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV6SinId.sol (207 líneas en total).

```
 4 // Lógica común: lotes y cadena de custodia.
 5 abstract contract GestionV6SinId {
 6     error NombreVacio();
 7     error TransicionInvalida();
 8     error NoDestinatario();
 9     error NoAutorizado();
10     error FabricanteVacio();
11     error TransferenciaPendiente();
12     error SinTransferencia();
13     error YaInactivo();
14     error DestinoInvalido();
15     error NoLaboratorio();
16     error MovimientoInexistente();
17     error NoCustodio();
18     error LoteInactivo();
19     error LoteInexistente();
20     error CustodioInconsistente();
```

contracts/gas-lab/GestionV7LotePacked.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV7LotePacked.sol (207 líneas en total).

```
 4 // Lógica común: lotes y cadena de custodia.
 5 abstract contract GestionV7LotePacked {
 6     error NombreVacio();
 7     error TransicionInvalida();
 8     error NoDestinatario();
 9     error NoAutorizado();
10     error FabricanteVacio();
11     error TransferenciaPendiente();
12     error SinTransferencia();
13     error YaInactivo();
14     error DestinoInvalido();
15     error NoLaboratorio();
16     error MovimientoInexistente();
17     error NoCustodio();
18     error LoteInactivo();
19     error LoteInexistente();
20     error CustodioInconsistente();
```

contracts/gas-lab/GestionV8HistorialPacked.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV8HistorialPacked.sol (207 líneas en total).

```
// Lógica común: lotes y cadena de custodia.
abstract contract GestionV8HistorialPacked {
    error NombreVacio();
    error TransicionInvalida();
    error NoDestinatario();
    error NoAutorizado();
    error FabricanteVacio();
    error TransferenciaPendiente();
    error SinTransferencia();
    error YaInactivo();
    error DestinoInvalido();
    error NoLaboratorio();
    error MovimientoInexistente();
    error NoCustodio();
    error LoteInactivo();
    error LoteInexistente();
    error CustodioInconsistente();
```

contracts/gas-lab/GestionV9TransferenciaCompacta.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/GestionV9TransferenciaCompacta.sol (209 líneas en total).

```
// Lógica común: lotes y cadena de custodia.
abstract contract GestionV9TransferenciaCompacta {
    error NombreVacio();
    error TransicionInvalida();
    error NoDestinatario();
    error NoAutorizado();
    error FabricanteVacio();
    error TransferenciaPendiente();
    error SinTransferencia();
    error YaInactivo();
    error DestinoInvalido();
    error NoLaboratorio();
    error MovimientoInexistente();
    error NoCustodio();
    error LoteInactivo();
    error LoteInexistente();
    error CustodioInconsistente();
```

contracts/gas-lab/PharmaBase.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/PharmaBase.sol (72 líneas en total).

```
// Versión didáctica: permisos programados manualmente.
contract PharmaBase is GestionBase {
    address public immutable administrador;
    bool public paused;

    enum Rol {

    event RolAsignado(address indexed usuario, Rol rol);
    event Paused(address indexed usuario);
    event Unpaused(address indexed usuario);

    constructor() {
        administrador = msg.sender;
    }

    modifier soloAdministrador() {
        require(msg.sender == administrador, "No eres el administrador");
        _;
    }

    // El administrador asigna un único rol por dirección.
    function asignarRol(address usuario, Rol rol) external soloAdministrador {
        require(usuario != address(0), "Direccion invalida");
        require(usuario != administrador, "El administrador no puede tener rol");
```

contracts/gas-lab/PharmaGasOptimizada.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/PharmaGasOptimizada.sol (80 líneas en total).

```
 6 // Versión didáctica: permisos programados manualmente.
 7 contract PharmaGasOptimizada is GestionGasOptimizada {
 8     error NoAdministrador();
 9     error DireccionInvalida();
10     error AdministradorSinRol();
11     error YaPausado();
12     error NoPausado();
13     error ContratoPausado();
14 
15     address public immutable administrador;
16     bool public paused;
17 
25 
26     event RolAsignado(address indexed usuario, Rol rol);
27     event Paused(address indexed usuario);
28     event Unpaused(address indexed usuario);
29 
30     constructor() {
31         administrador = msg.sender;
32     }
33 
34     modifier soloAdministrador() {
35         if (msg.sender != administrador) revert NoAdministrador();
36         _;
```

contracts/gas-lab/PharmaOZV0Base.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/PharmaOZV0Base.sol (53 líneas en total).

```
 8 // Versión con controles de seguridad de OpenZeppelin.
 9 contract PharmaOZV0Base is
10     GestionV0Base,
11     AccessControl,
12     Pausable
13 {
20 
21     constructor() {
22         administrador = msg.sender;
23         _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
24     }
25 
26     function pausar() external onlyRole(DEFAULT_ADMIN_ROLE) {
27         _pause();
28     }
29     function reanudar() external onlyRole(DEFAULT_ADMIN_ROLE) {
30         _unpause();
31     }
32 
33     function _esLaboratorio(address usuario) internal view override returns (bool) {
34         return hasRole(LABORATORIO_ROLE, usuario);
35     }
36 
37     function _esAdministrador(address usuario) internal view override returns (bool) {
```

contracts/gas-lab/PharmaOZV10PackingCombinado.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/PharmaOZV10PackingCombinado.sol (53 líneas en total).

```
 8 // Versión con controles de seguridad de OpenZeppelin.
 9 contract PharmaOZV10PackingCombinado is
10     GestionV10PackingCombinado,
11     AccessControl,
12     Pausable
13 {
20 
21     constructor() {
22         administrador = msg.sender;
23         _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
24     }
25 
26     function pausar() external onlyRole(DEFAULT_ADMIN_ROLE) {
27         _pause();
28     }
```

```
29      function reanudar() external onlyRole(DEFAULT_ADMIN_ROLE) {
30          _unpause();
31      }
32
33      function _esLaboratorio(address usuario) internal view override returns (bool) {
34          return hasRole(LABORATORIO_ROLE, usuario);
35      }
36
37      function _esAdministrador(address usuario) internal view override returns (bool) {
```

contracts/gas-lab/PharmaOZV11SoloEventos.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/PharmaOZV11SoloEventos.sol (53 líneas en total).

```
 8 // Versión con controles de seguridad de OpenZeppelin.
 9 contract PharmaOZV11SoloEventos is
10      GestionV11SoloEventos,
11      AccessControl,
12      Pausable
13 {
20
21      constructor() {
22          administrador = msg.sender;
23          _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
24      }
25
26      function pausar() external onlyRole(DEFAULT_ADMIN_ROLE) {
27          _pause();
28      }
29      function reanudar() external onlyRole(DEFAULT_ADMIN_ROLE) {
30          _unpause();
31      }
32
33      function _esLaboratorio(address usuario) internal view override returns (bool) {
34          return hasRole(LABORATORIO_ROLE, usuario);
35      }
36
37      function _esAdministrador(address usuario) internal view override returns (bool) {
```

contracts/gas-lab/PharmaOZV12EventosCompacta.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/PharmaOZV12EventosCompacta.sol (53 líneas en total).

```
 8 // Versión con controles de seguridad de OpenZeppelin.
 9 contract PharmaOZV12EventosCompacta is
10      GestionV12EventosCompacta,
11      AccessControl,
12      Pausable
13 {
20
21      constructor() {
22          administrador = msg.sender;
23          _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
24      }
25
26      function pausar() external onlyRole(DEFAULT_ADMIN_ROLE) {
27          _pause();
28      }
29      function reanudar() external onlyRole(DEFAULT_ADMIN_ROLE) {
30          _unpause();
31      }
32
33      function _esLaboratorio(address usuario) internal view override returns (bool) {
34          return hasRole(LABORATORIO_ROLE, usuario);
35      }
36
37      function _esAdministrador(address usuario) internal view override returns (bool) {
```

contracts/gas-lab/PharmaOZV1Roles.sol

Extracto del archivo E5/PharmaChain-Ethereum/contracts/gas-lab/PharmaOZV1Roles.sol (53 líneas en total).

```
 8 // Versión con controles de seguridad de OpenZeppelin.
 9 contract PharmaOZV1Roles is
10      GestionV1Roles,
11      AccessControl,
12      Pausable
```

```
13 {
20
21     constructor() {
22         administrador = msg.sender;
23         _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
24     }
25
26     function pausar() external onlyRole(DEFAULT_ADMIN_ROLE) {
27         _pause();
28     }
29     function reanudar() external onlyRole(DEFAULT_ADMIN_ROLE) {
30         _unpause();
31     }
32
33     function _esLaboratorio(address usuario) internal view override returns (bool) {
34         return hasRole(LABORATORIO_ROLE, usuario);
35     }
36
37     function _esAdministrador(address usuario) internal view override returns (bool) {
```

Las versiones basadas solo en eventos pueden modificar la capacidad de consultar el historial directamente desde el contrato.

8. E5 — Pruebas automatizadas

Suite	Propósito	Existe
ComparacionPermisos.test.js	Roles manuales frente a AccessControl	Sí
FourVariantsParity.test.js	Equivalencia entre cuatro implementaciones	Sí
FuncionalidadCompartida.test.js	Operaciones comunes y restricciones	Sí
GasComparison.test.js	Comparación de consumo de gas	Sí
GasExperiments.test.js	Microoptimizaciones y coste de despliegue	Sí
GasLabExtended.test.js	Escenarios de GasLab V0–V12	Sí
GasLabInvariants.test.js	Invariantes funcionales y de seguridad	Sí
HistorialCustodia.test.js	Historial de custodia	Sí
LoteSinId.test.js	Variantes sin ID redundante	Sí
OptimizedParity.test.js	Paridad entre variantes optimizadas	Sí
PharmaGasLab.test.js	Pruebas experimentales de gas	Sí
VariantCatalog.test.cjs	Catálogo de variantes	Sí

ComparacionPermisos.test.js

Caso declarado: Diferencias de permisos

Caso declarado: manual: un usuario solo tiene un rol

Caso declarado: manual: un atacante no puede asignar roles

Caso declarado: OpenZeppelin: permite conceder y revocar un rol

Caso declarado: OpenZeppelin: un atacante no puede conceder roles

Caso declarado: OpenZeppelin: una cuenta puede tener varios roles

Extracto del archivo E5/PharmaChain-Ethereum/test/ComparacionPermisos.test.js (55 líneas en total).

```
 3
 4 describe("Diferencias de permisos", function () {
 5     let ethers, admin, lab, otra;
 6     beforeEach(async function () {
 7         const conexion = await network.connect();
 8         ethers = conexion.ethers;
 9         [admin, lab, otra] = await ethers.getSigners();
10     });
```

```
11
12      it("manual: un usuario solo tiene un rol", async function () {
13          const F = await ethers.getContractFactory("PharmaChainRolesManuales");
14          const c = await F.deploy();
15          await c.asignarRol(lab.address, 1);
16          expect(await c.roles(lab.address)).to.equal(1n);
20
```

FourVariantsParity.test.js

Caso declarado: Integracion completa: paridad de comportamiento V0–V7

Caso declarado: ejecuta el mismo escenario y compara estados e historiales en las ocho variantes

Extracto del archivo E5/PharmaChain-Ethereum/test/FourVariantsParity.test.js (151 líneas en total).

```
 3
 4 // Use fully qualified names: this repository contains duplicate Solidity contract names.
 5 const VARIANTS = [
 6   { id: "V0", kind: "manual", fqn: "contracts/PharmaChainRolesManuales.sol:PharmaChainRolesMan
 7   { id: "V1", kind: "oz", fqn: "contracts/PharmaChainRolesOpenZeppelin.sol:PharmaChainRolesOpe
 8   { id: "V2", kind: "manual", fqn: "contracts/PharmaChainRolesManualesOptimized.sol:PharmaChai
15
16 async function mustFail(operation, label) {
17   let failed = false;
18   try {
19     const tx = await operation();
20     if (tx?.wait) await tx.wait();
26
27 async function setup(ethers, variant, actors) {
28   const [admin, lab, distributor, pharmacy] = actors;
```

FuncionalidadCompartida.test.js

Caso declarado: Comportamiento común: ${nombre}

Caso declarado: empieza sin lotes

Caso declarado: registra y consulta lotes con todos sus datos

Caso declarado: crea IDs consecutivos

Caso declarado: rechaza nombres y fabricantes vacíos

Caso declarado: impide registrar al atacante

Caso declarado: rechaza lotes inexistentes

Caso declarado: emite evento de registro

Caso declarado: la fecha corresponde al bloque

Caso declarado: la propuesta no cambia al custodio

Caso declarado: completa laboratorio -> distribuidor -> farmacia

Caso declarado: rechaza salto directo laboratorio -> farmacia

Caso declarado: rechaza aceptar por un tercero

Caso declarado: cancela una transferencia sin cambiar al custodio

Caso declarado: impide cancelar a un atacante

Caso declarado: la pausa bloquea el registro y se puede reanudar

Caso declarado: desactiva lotes y cancela su transferencia pendiente

Caso declarado: solo el administrador puede desactivar

Caso declarado: registra el gas utilizado

Extracto del archivo E5/PharmaChain-Ethereum/test/FuncionalidadCompartida.test.js (161 líneas en total).

```
 5 for (const nombre of ["PharmaChainRolesManuales", "PharmaChainRolesOpenZeppelin"]) {
 6     describe(`Comportamiento común: ${nombre}`, function () {
 7         let ethers, contrato, admin, lab, dist, farmacia, atacante;
```

```
 8
 9         beforeEach(async function () {
10             const conexion = await network.connect();
11             ethers = conexion.ethers;
12             [admin, lab, dist, farmacia, atacante] = await ethers.getSigners();
13             const Fabrica = await ethers.getContractFactory(nombre);
27
28         async function registrar(nombreLote = "Paracetamol", fabricante = "Laboratorio Galicia
29             return contrato.connect(lab).registrarLote(nombreLote, fabricante);
30         }
31
32         it("empieza sin lotes", async function () {
```

GasComparison.test.js

Caso declarado: PharmaChain gas ${labels[i]}

Caso declarado: ejecuta registro, transferencia, cancelacion y desactivacion

Caso declarado: desactiva con transferencia pendiente

Caso declarado: mantiene permisos, validaciones y pausa

Caso declarado: registra varios lotes para medir ranuras nuevas

Extracto del archivo E5/PharmaChain-Ethereum/test/GasComparison.test.js (97 líneas en total).

```
11
12 async function deploy(index) {
13   const [admin, lab, dist, pharmacy, attacker] = await ethers.getSigners();
14   const factory = await ethers.getContractFactory(contracts[index]);
15   const contract = await factory.deploy();
16   await contract.waitForDeployment();
17   const code = await ethers.provider.getCode(await contract.getAddress());
18   report.push({ version: labels[index], operacion: 'bytecode_bytes', valor: (code.length - 2)
19   for (const [who, role] of [[lab, 1], [dist, 2], [pharmacy, 3]]) {
20     await (await contract.connect(admin).asignarRol(who.address, role)).wait();
21   }
22   return { contract, admin, lab, dist, pharmacy, attacker };
23 }
24 async function mustRevert(action) {
25   let failed = false;
```

GasExperiments.test.js

Caso declarado: Gas Lab: equivalencia funcional y mediciones

Caso declarado: variante ${variant}

Extracto del archivo E5/PharmaChain-Ethereum/test/GasExperiments.test.js (68 líneas en total).

```
10 const asNum = (x) => Number(x);
11 async function measured(variant, operation, txPromise) {
12   const tx = await txPromise;
13   const receipt = await tx.wait();
14   results.push({ variant, operation, gas: asNum(receipt.gasUsed) });
15   return receipt;
16 }
17 async function fixture(variant) {
18   const [admin, lab, distributor, pharmacy, attacker] = await ethers.getSigners();
19   const factory = await ethers.getContractFactory(`GasLab${variant}`);
20   const c = await factory.deploy();
21   const deployment = await c.deploymentTransaction().wait();
30
31 describe('Gas Lab: equivalencia funcional y mediciones', function () {
32   for (const variant of VARIANTS) {
```

GasLabExtended.test.js

Caso declarado: GasLab EXT ${family} ${version}

Caso declarado: registro: ID, eventos, textos, timestamp, permisos, getters y gas

Caso declarado: custodia completa: proponer, aceptar, cancelar, roles y gas

Caso declarado: cancelación administrativa y desactivación con transferencia pendiente

Caso declarado: pausa, reanudación y cancelación administrativa durante pausa

Caso declarado: registro en serie y longitudes límite para cadenas

Caso declarado: asignación de roles, revocación y seguridad de autorización

Extracto del archivo E5/PharmaChain-Ethereum/test/GasLabExtended.test.js (197 líneas en total).

```
const key = (family,v,operation) => `${family}:${v}:${operation}`;
function record(family, version, operation, gas, kind='tx') {
  report.push({family,version,operation,kind,gas:gas.toString()});
}
async function success(p) {
  const tx=await p;
  const r=await tx.wait();
  expect(r.status).to.equal(1);
  return r;
}
async function measure(f,v,operation,p) {
  const r=await success(p);
  record(f,v,operation,r.gasUsed);
  return r;
}
```

GasLabInvariants.test.js

Caso declarado: Invariants ${family} ${version}

Caso declarado: no puede transferir ni reactivar un lote desactivado; preserva propiedad

Caso declarado: revocar o cambiar el rol del destino invalida una aceptación pendiente

Caso declarado: secuencias deterministas: nunca acepta un destino no autorizado

Extracto del archivo E5/PharmaChain-Ethereum/test/GasLabInvariants.test.js (68 líneas en total).

```
const mine=async tx=>{const r=await(await tx).wait();expect(r.status).to.equal(1);return r;};
async function reject(fn){let fail=false;try{await mine(fn());}catch{fail=true;}expect(fail).t
async function setup(f,v){
 const name=f==='Manual'?`Pharma${v}`:`PharmaOZ${v}`;
 const c=await(await ethers.getContractFactory(name)).deploy();await c.waitForDeployment();
 for(let i=1;i<=3;i++){
  if(f==='Manual')await mine(c.asignarRol(users[i].address,i));
}
function emitted(c,receipt,event){
 return receipt.logs.flatMap(log=>{try{return [c.interface.parseLog(log)];}catch{return [];}})
}
function random(seed){let x=seed>>>0;return ()=>{x=(Math.imul(x,1664525)+1013904223)>>>0;retur
before(async()=>{users=await ethers.getSigners();});
for(const family of ['Manual','OZ']) for(const version of versions){
 describe(`Invariants ${family} ${version}`,function(){
```

HistorialCustodia.test.js

Caso declarado: Historial de custodia - ${version}

Caso declarado: Registra el primer movimiento al crear un lote

Caso declarado: Guarda la propuesta sin cambiar el custodio

Caso declarado: Conserva el historial completo hasta la farmacia

Caso declarado: Registra una cancelacion sin cambiar el custodio

Caso declarado: Rechaza consultar movimientos inexistentes

Extracto del archivo E5/PharmaChain-Ethereum/test/HistorialCustodia.test.js (115 líneas en total).

```
for (const version of versiones) {
    describe(`Historial de custodia - ${version}`, function () {
        let ethers;
        let contrato;
        let admin;
        let laboratorio;

        beforeEach(async function () {
            const conexion = await network.connect();
            ethers = conexion.ethers;

            [admin, laboratorio, distribuidor, farmacia] = await ethers.getSigners();

```

```
39          it("Registra el primer movimiento al crear un lote", async function () {
40              await contrato.connect(laboratorio).registrarLote("Paracetamol", "Laboratorio Gali
```

LoteSinId.test.js

Caso declarado: PharmaChain - Lote sin ID redundante

Caso declarado: registra un lote con identificador consecutivo

Caso declarado: no devuelve un campo ID redundante

Caso declarado: mantiene IDs consecutivos

Caso declarado: mantiene la transferencia de custodia

Caso declarado: mantiene el historial de movimientos

Caso declarado: rechaza registros sin rol de laboratorio

Caso declarado: mide el gas de varios registros

Extracto del archivo E5/PharmaChain-Ethereum/test/LoteSinId.test.js (131 líneas en total).

```
 3
 4 describe("PharmaChain - Lote sin ID redundante", function () {
 5   let ethers;
 6   let contrato;
 7   let admin, laboratorio, distribuidor, farmacia;
 8
 9   beforeEach(async function () {
10     ({ ethers } = await hre.network.connect());
11
12     [admin, laboratorio, distribuidor, farmacia] =
13       await ethers.getSigners();
34
35   async function registrar(nombre = "Paracetamol") {
36     const tx = await contrato
37       .connect(laboratorio)
```

OptimizedParity.test.js

Caso declarado: Seguridad, custodia e historial — ${name}

Caso declarado: mantiene el administrador y el contador inicial

Caso declarado: registra y consulta datos e IDs consecutivos

Caso declarado: rechaza registros sin rol, nombre vacío o fabricante vacío

Caso declarado: rechaza consultas inexistentes

Caso declarado: mantiene historial completo tras transferencias

Caso declarado: rechaza transición directa laboratorio a farmacia y terceros

Caso declarado: permite cancelar por origen y por administrador, sin alterar propietario

Caso declarado: desactiva lote y elimina propuesta pendiente

Caso declarado: bloquea operaciones protegidas cuando está pausado y permite reanudar

Extracto del archivo E5/PharmaChain-Ethereum/test/OptimizedParity.test.js (114 líneas en total).

```
 8
 9 async function fixture(ethers, name, kind, signers) {
10     const [admin, lab, dist, pharmacy] = signers;
11     const factory = await ethers.getContractFactory(name);
12     const contract = await factory.deploy();
13     await contract.waitForDeployment();
14     if (kind === "manual") {
15         await (await contract.asignarRol(lab.address, 1)).wait();
16         await (await contract.asignarRol(dist.address, 2)).wait();
17         await (await contract.asignarRol(pharmacy.address, 3)).wait();
18     } else {
19         await (await contract.grantRole(await contract.LABORATORIO_ROLE(), lab.address)).wait(
20         await (await contract.grantRole(await contract.DISTRIBUIDOR_ROLE(), dist.address)).wai
21         await (await contract.grantRole(await contract.FARMACIA_ROLE(), pharmacy.address)).wai
22     }
```

PharmaGasLab.test.js

Caso declarado: GasLab ${version}

Caso declarado: recorre cadena de custodia y comprueba historial

Caso declarado: desactiva con transferencia pendiente

Caso declarado: rechaza acciones inválidas y conserva la pausa

Caso declarado: mide registros sucesivos, cadenas largas y cancelación administrativa

Extracto del archivo E5/PharmaChain-Ethereum/test/PharmaGasLab.test.js (158 líneas en total).

```
13
14 function record(version, operation, gas, kind='transaction') {
15   measurements.push({version,operation,kind,gas:gas.toString()});
16 }
17 async function receipt(txPromise) {
18   const tx = await txPromise;
19   const r = await tx.wait();
20   expect(r.status).to.equal(1);
21   return r;
22 }
23 async function measured(version, operation, txPromise) {
24   const r = await receipt(txPromise);
25   record(version,operation,r.gasUsed);
26   return r;
27 }
```

VariantCatalog.test.cjs

Caso declarado: identificadores únicos y contratos fuente disponibles

Caso declarado: cuatro variantes por modelo de permisos

Extracto del archivo E5/PharmaChain-Ethereum/test/VariantCatalog.test.cjs (16 líneas en total).

```
1 const {test}=require('node:test');
2 const assert=require('node:assert/strict');
3 const fs=require('node:fs');
4 const path=require('node:path');
5 const variants=require('../lab/variants.json');
6 test('identificadores únicos y contratos fuente disponibles',()=>{
7  assert.equal(new Set(variants.map(v=>v.id)).size,variants.length);
8  for(const v of variants){
9   assert.ok(v.title&&v.description&&v.optimization);
10   assert.ok(fs.existsSync(path.join(__dirname,'..','contracts',v.id+'.sol')));
12 });
13 test('cuatro variantes por modelo de permisos',()=>{
14  assert.equal(variants.filter(v=>v.kind==='manual').length,4);
15  assert.equal(variants.filter(v=>v.kind==='openzeppelin').length,4);
16 });
```

La presencia de un test no demuestra que haya pasado. Para validar resultados es necesario ejecutarlo y conservar el log.

9. E5 — Resultados de gas y benchmarks

gas-reports/comparison-extended.csv

Filas registradas: 576.

family	version	operation	baseline
Manual	V10PackingCombinado	accept_dist_pharm	113450
Manual	V10PackingCombinado	accept_lab_dist	113202
Manual	V10PackingCombinado	cancel_admin	98899
Manual	V10PackingCombinado	cancel_dist_pharm	98871
Manual	V10PackingCombinado	cancel_while_paused	98899

family	version	operation	baseline
Manual	V10PackingCombinado	deactivate_pending	33948
Manual	V10PackingCombinado	deploy	1705592
Manual	V10PackingCombinado	pause	44733
Manual	V10PackingCombinado	propose_dist_pharm	154322
Manual	V10PackingCombinado	propose_lab_dist	154062
Manual	V10PackingCombinado	register_first	255742
Manual	V10PackingCombinado	register_len_1	255574
Manual	V10PackingCombinado	register_len_128	329982
Manual	V10PackingCombinado	register_len_256	421855
Manual	V10PackingCombinado	register_len_31	238834
Manual	V10PackingCombinado	register_len_32	261086
Manual	V10PackingCombinado	register_len_33	283641
Manual	V10PackingCombinado	register_len_64	284051
Manual	V10PackingCombinado	repropose_dist_pharm	154322
Manual	V10PackingCombinado	resume	22845
Manual	V10PackingCombinado	role_1	45881
Manual	V10PackingCombinado	role_2	45869
Manual	V10PackingCombinado	role_3	45881
Manual	V10PackingCombinado	runtime_bytes	7650
Manual	V11SoloEventos	accept_dist_pharm	113450
Manual	V11SoloEventos	accept_lab_dist	113202
Manual	V11SoloEventos	cancel_admin	98899
Manual	V11SoloEventos	cancel_dist_pharm	98871
Manual	V11SoloEventos	cancel_while_paused	98899
Manual	V11SoloEventos	deactivate_pending	33948

gas-reports/gas-experiments.csv

Filas registradas: 8.

variant	operation	gas
Base	deploy	1730665
Base	runtime_bytecode_bytes	7766
Cache	deploy	1719194
Cache	runtime_bytecode_bytes	7713
Errors	deploy	1678412
Errors	runtime_bytecode_bytes	7524
CacheErrors	deploy	1666876
CacheErrors	runtime_bytecode_bytes	7471

gas-reports/pharmachain-default.csv

Filas registradas: 287.

version	operation	kind	value
V0Base	despliegue	deployment	1705592
V0Base	runtime_bytes	bytes	7650
V0Base	asignar_rol_1	transaction	45881
V0Base	asignar_rol_2	transaction	45869
V0Base	asignar_rol_3	transaction	45881
V0Base	registro_corto	transaction	255742
V0Base	propuesta_lab_dist	transaction	154062
V0Base	aceptacion_lab_dist	transaction	113202
V0Base	propuesta_dist_farm	transaction	154322
V0Base	cancelacion	transaction	98871
V0Base	desactivar_sin_pendiente	transaction	27536
V0Base	despliegue	deployment	1705592
V0Base	runtime_bytes	bytes	7650
V0Base	asignar_rol_1	transaction	45881
V0Base	asignar_rol_2	transaction	45869
V0Base	asignar_rol_3	transaction	45881
V0Base	desactivar_con_pendiente	transaction	33948
V0Base	despliegue	deployment	1705592
V0Base	runtime_bytes	bytes	7650
V0Base	asignar_rol_1	transaction	45881
V0Base	asignar_rol_2	transaction	45869
V0Base	asignar_rol_3	transaction	45881
V0Base	pausar	transaction	44733
V0Base	reanudar	transaction	22845
V0Base	despliegue	deployment	1705592
V0Base	runtime_bytes	bytes	7650
V0Base	asignar_rol_1	transaction	45881
V0Base	asignar_rol_2	transaction	45869
V0Base	asignar_rol_3	transaction	45881
V0Base	registro_serie_1	transaction	255778

gas-reports/pharmachain-extended.csv

Filas registradas: 1274.

family	version	operation	kind
Manual	V0Base	deploy	deployment
Manual	V0Base	runtime_bytes	bytes

family	version	operation	kind
Manual	V0Base	role_1	tx
Manual	V0Base	role_2	tx
Manual	V0Base	role_3	tx
Manual	V0Base	register_first	tx
Manual	V0Base	deploy	deployment
Manual	V0Base	runtime_bytes	bytes
Manual	V0Base	role_1	tx
Manual	V0Base	role_2	tx
Manual	V0Base	role_3	tx
Manual	V0Base	propose_lab_dist	tx
Manual	V0Base	accept_lab_dist	tx
Manual	V0Base	propose_dist_pharm	tx
Manual	V0Base	cancel_dist_pharm	tx
Manual	V0Base	repropose_dist_pharm	tx
Manual	V0Base	accept_dist_pharm	tx
Manual	V0Base	deploy	deployment
Manual	V0Base	runtime_bytes	bytes
Manual	V0Base	role_1	tx
Manual	V0Base	role_2	tx
Manual	V0Base	role_3	tx
Manual	V0Base	cancel_admin	tx
Manual	V0Base	deactivate_pending	tx
Manual	V0Base	deploy	deployment
Manual	V0Base	runtime_bytes	bytes
Manual	V0Base	role_1	tx
Manual	V0Base	role_2	tx
Manual	V0Base	role_3	tx
Manual	V0Base	pause	tx

gas-reports/pharmachain-gas.csv

Filas registradas: 32.

version	operacion	valor
base	bytecode_bytes	7650
base	registro_1	255742
base	propuesta_lab_dist	154062
base	aceptacion_lab_dist	113202
base	propuesta_dist_farm	154322
base	cancelacion_dist	98871

version	operacion	valor
base	desactivar_sin_pendiente	27536
base	bytecode_bytes	7650
base	desactivar_con_pendiente	33948
base	bytecode_bytes	7650
base	bytecode_bytes	7650
base	registro_1_serie	255778
base	registro_2_serie	238678
base	registro_3_serie	238678
base	registro_4_serie	238678
base	registro_5_serie	238678
optimizada	bytecode_bytes	6459
optimizada	registro_1	255307
optimizada	propuesta_lab_dist	153814
optimizada	aceptacion_lab_dist	113073
optimizada	propuesta_dist_farm	153886
optimizada	cancelacion_dist	98875
optimizada	desactivar_sin_pendiente	27473
optimizada	bytecode_bytes	6459
optimizada	desactivar_con_pendiente	33898
optimizada	bytecode_bytes	6459
optimizada	bytecode_bytes	6459
optimizada	registro_1_serie	255343
optimizada	registro_2_serie	238243
optimizada	registro_3_serie	238243

reports/gas-comparison.csv

Filas registradas: 14.

arquitectura	operacion	original	optimizado
Manuales	despliegue	1730665	1705592
Manuales	registrarLote	255910	255910
Manuales	proponerTransferencia	154062	154062
Manuales	aceptarTransferencia	113002	113202
Manuales	proponerParaCancelar	154322	154322
Manuales	cancelarTransferencia	98807	98871
Manuales	desactivarLote	27536	27536
OpenZeppelin	despliegue	1761337	1736264
OpenZeppelin	registrarLote	256076	256076
OpenZeppelin	proponerTransferencia	154260	154260

arquitectura	operacion	original	optimizado
OpenZeppelin	aceptarTransferencia	113243	113443
OpenZeppelin	proponerParaCancelar	156596	156596
OpenZeppelin	cancelarTransferencia	98895	98959
OpenZeppelin	desactivarLote	29806	29806

9.1. Comparativa por familia y variante

Familia	Versión	Deploy gas	Registro gas	Ahorro vs V0
Manual	V0Base	1,705,592	255,742	+0.00%
Manual	V1Roles	1,692,614	255,742	+0.00%
Manual	V2Errores	1,485,444	255,742	+0.00%
Manual	V3Storage	1,686,979	255,742	+0.00%
Manual	V4Unchecked	1,694,536	255,582	+0.06%
Manual	V5Combinada	1,442,863	255,582	+0.06%
Manual	V6SinId	1,429,226	233,428	+8.73%
Manual	V7LotePacked	1,448,469	189,274	+25.99%
Manual	V8HistorialPacked	1,441,596	231,155	+9.61%
Manual	V9TransferenciaCompacta	1,411,501	233,428	+8.73%
Manual	V10PackingCombinado	1,440,096	187,121	+26.83%
Manual	V11SoloEventos	1,282,803	166,748	+34.80%
Manual	V12EventosCompacta	1,284,334	122,594	+52.06%
OZ	V0Base	1,736,264	255,908	+0.00%
OZ	V1Roles	1,737,344	255,908	+0.00%
OZ	V2Errores	1,570,343	255,908	+0.00%
OZ	V3Storage	1,718,743	255,908	+0.00%
OZ	V4Unchecked	1,726,324	255,748	+0.06%
OZ	V5Combinada	1,540,696	255,748	+0.06%
OZ	V6SinId	1,527,071	233,594	+8.72%
OZ	V7LotePacked	1,546,309	189,440	+25.97%
OZ	V8HistorialPacked	1,539,448	231,321	+9.61%
OZ	V9TransferenciaCompacta	1,509,351	233,594	+8.72%
OZ	V10PackingCombinado	1,537,922	187,287	+26.81%
OZ	V11SoloEventos	1,373,158	166,885	+34.79%
OZ	V12EventosCompacta	1,374,670	122,731	+52.04%

Ahorro positivo indica reducción de gas en register_first respecto a V0Base de la misma familia. Deploy se muestra aparte.

10. Conclusiones y limitaciones

El gas de transacciones y el tamaño del bytecode son magnitudes diferentes.

Los resultados dependen del compilador, la configuración y el estado de la EVM.

Las variantes de roles manuales y OpenZeppelin pueden diferir funcionalmente.

La sustitución de historial persistente por eventos modifica las consultas on-chain.

Los datos de CSV son mediciones almacenadas, no ejecuciones realizadas por este generador.

11. Bibliografía y referencias de trabajo

Fuentes técnicas y educativas pertinentes al proyecto. Su inclusión no equivale a afirmar que cada una fue consultada directamente.

[1] TRD_Blockchain — repositorio del trabajo

Código, pruebas, documentación y resultados del proyecto.

https://github.com/ajlorenzo1315/TRD_Blockchain

[2] CryptoZombies — curso interactivo

Material de aprendizaje para el ejercicio E1.

https://cryptozombies.io/

[3] Solidity — documentación oficial

Lenguaje, estructuras, herencia, eventos y seguridad.

https://docs.soliditylang.org/

[4] Solidity — optimizador

Optimizaciones del compilador y generación de bytecode.

https://docs.soliditylang.org/en/latest/internals/optimizer.html

[5] Hardhat — documentación

Compilación, pruebas y redes de desarrollo.

https://hardhat.org/docs

[6] Ethereum.org — recursos para desarrolladores

Fundamentos de Ethereum y de los contratos inteligentes.

https://ethereum.org/developers/

[7] Ethereum.org — gas

Modelo de gas y costes de ejecución.

https://ethereum.org/en/developers/docs/gas/

[8] Remix IDE

Edición, compilación y pruebas de Solidity.

https://remix.ethereum.org/

[9] OpenZeppelin Contracts

Bibliotecas ERC20, ERC721 y contratos reutilizables.

https://docs.openzeppelin.com/contracts/

[10] OpenZeppelin — control de acceso

Control de acceso basado en roles.

https://docs.openzeppelin.com/contracts/5.x/access-control

[11] RareSkills

Lecturas complementarias de EVM y optimización.

https://www.rareskills.io/

[12] Loom Network — GitHub

Repositorio del ecosistema asociado a CryptoZombies.

https://github.com/loomnetwork